福大智联_系统设计说明书

项目名称： 考研英语领域智能体助学系统（Kaoyan English Agent）

文档编号： FZU-SE-2026-KEA-SDD-001

文档版本： V1.0

编制日期： 2026-11-09

编制单位： 福大智联团队

课程名称： 2026 秋软件工程实践

代码基线： `main @ 0688e14`（2026-11-08）

规范依据： GB/T 8567-2006《计算机软件文档编制规范》

目录

- 1 引言
- 1.1 编写目的
- 1.2 背景
- 1.3 术语与缩写词定义
- 1.4 参考资料
- 2 系统总体设计
- 2.1 需求规定回顾
- 2.2 运行环境
- 2.3 基本设计概念与分层处理流程
- 2.4 系统结构
- 2.5 人工处理过程与尚未解决的问题
- 3 领域智能体专项设计
- 3.1 智能体与业务系统集成架构
- 3.2 提示词工程体系
- 3.3 领域工具集设计
- 3.4 多步任务工作流与轨迹追踪
- 4 接口设计
- 4.1 用户接口
- 4.2 外部接口

- 4.3 内部接口
- 5 模块详细层级设计
- 5.1 学情仪表盘模块
- 5.2 Agent 智能对话模块
- 5.3 长难句解析模块
- 5.4 阅读选项分析模块
- 5.5 生词本模块
- 5.6 领域知识库模块
- 5.7 真题库与后台导入模块
- 5.8 学情诊断报告模块
- 6 系统出错处理与安全保密设计
- 6.1 LLM 接口超时、限流降级策略
- 6.2 数据校验、异常恢复与备份维护设计
- 附录 A 接口清单总表
- 附录 B HTTP 状态码使用约定
- 附录 C 源码与文档索引

---

1 引言

1.1 编写目的

本文档是"考研英语领域智能体助学系统（Kaoyan English Agent）"的**系统设计说明书**，用于在需求分析完成之后、编码实现已具备可运行基线的前提下，自顶向下说明系统的总体结构、分层职责、领域智能体（Domain Agent）的集成方式、内部与外部接口、各业务模块的详细处理逻辑，以及出错处理与安全保密设计。

本文档的预期读者包括：

读者	使用方式
课程指导教师与评审助教	依据本文档核查设计是否覆盖全部需求、是否符合 GB/T 8567-2006 的结构要求、设计与代码是否一致
团队成员（前端、后端、文档、测试）	以本文档作为联调、维护与后续迭代的统一设计基线
后续接手者（学弟学妹或下一届团队）	通过本文档与附录源码索引快速建立系统心智模型
答辩现场听众	通过第 2、3、4 章理解"这不是聊天框套壳，而是工具、状态、数据库协同的领域系统"

本文档的编写原则是：**所有描述必须能追溯到仓库中的真实文件、真实函数或真实数据表。**凡文档与现有历史材料口径不一致之处，以当前代码基线为准，并在正文中显式标注差异来源。

1.2 背景

1.2.1 项目名称

- 中文名称：考研英语领域智能体助学系统
- 英文名称：Kaoyan English Agent
- 产品别名（前端品牌名）：研英智学
- 代码仓库：`kaoyan-english-agent`
- 仓库远程地址：`https://github.com/H3HN/kaoyan-english-agent.git`

1.2.2 项目开发者

本项目由福州大学 2026 秋软件工程实践课程团队"福大智联"共同开发，团队共 13 人：

序号	学号	姓名
1	102402125	黄俊翔
2	102402126	黄炜杰
3	102402127	金若曦
4	102402143	越长智
5	102402124	洪豐幀
6	102402134	王世豪
7	102402133	谭段昊
8	102402132	邱福铭
9	102402135	王伟浩
10	102402119	陈封
11	102402121	陈韦融
12	102402122	邓杰
13	102302113	王泽炜

文档编制与统稿：黄俊翔。代码实现按前端、后端、数据、文档四条工作线协作完成，具体提交记录以 Git 历史为准。

1.2.3 用户

系统的直接用户与间接用户如下：

角色	说明	使用界面
备考学生（主要用户）	考研英语一/英语二考生，使用长难句拆解、阅读训练、对话、单词本、报告等功能	`web/` 的 8 个页面
团队内容维护者	负责真题 JSON 草稿导入、解析生成、人工审核与发布、词库与陷阱规则维护	`server/scripts/` 命令行
系统管理员	配置 LLM API Key、启动前后端服务、备份数据库、恢复快照	服务器本地终端

当前系统为**单用户本地应用**：没有登录、账号体系与多用户数据隔离，`user_vocab_book` 与 `practice_records` 均无 `user_id` 字段。所有学习记录属于同一台机器上的同一使用者。

1.2.4 运行中心

- 开发与演示运行中心：团队成员本地计算机（Windows 11 / macOS / Linux）。
- 服务进程：FastAPI 进程监听 `8000` 端口；Vite 开发服务器监听 `5173` 端口。
- 数据位置：`server/data/app.db`（SQLite 单文件数据库），随仓库分发的题库快照为 `server/data/demo_snapshot.json`。
- 外部依赖：DeepSeek 开放平台（OpenAI 兼容接口），可选配置；未配置时系统以确定性模式运行。

1.3 术语与缩写词定义

本节按 GB/T 8567-2006 要求，对全文出现的英文缩写**先行定义**。为避免误导，表中同时标注"本项目是否采用"；未采用的术语仅作为行业背景说明，不得在正文中据此宣称系统具备该能力。

1.3.1 通用软件工程术语

缩写 / 术语	英文全称	中文含义	本项目采用情况
GB/T	GuoBiao / Recommended Standard（国家标准/推荐性标准）	中国国家标准体系中的推荐性标准前缀	采用，本文档依据 GB/T 8567-2006
SDD	Software Design Description	软件设计说明书，即本文档	采用
SRS	Software Requirements Specification	软件需求规格说明书，本项目对应 `docs/需求文档.md`	采用
UML	Unified Modeling Language	统一建模语言，用于用例图、类图、活动图、时序图、通信图	采用，见图集文档
ER	Entity-Relationship	实体—关系模型，用于数据库概念设计	采用
UI	User Interface	用户界面	采用
UX	User Experience	用户体验	采用
MVP	Minimum Viable Product	最小可行产品，本项目指 `kaoyan_agent/` 早期 Streamlit 版本	采用（历史形态）

缩写 / 术语	英文全称	中文含义	本项目采用情况
CLI	Command Line Interface	命令行界面，本项目题库维护入口	采用
IDE	Integrated Development Environment	集成开发环境	采用（开发工具）
OS	Operating System	操作系统	采用
PDF	Portable Document Format	便携式文档格式，真题原始载体	采用（导入来源）
OCR	Optical Character Recognition	光学字符识别，扫描版 PDF 转文本	**未实现**，当前仅支持文本型 PDF
GB	Gigabyte	存储容量单位，1 GB = 1024 MB	采用（容量限制说明）

1.3.2 架构与接口术语

缩写 / 术语	英文全称	中文含义	本项目采用情况
API	Application Programming Interface	应用程序编程接口	采用，后端提供 HTTP API
REST	Representational State Transfer	表述性状态转移，一种网络应用架构风格	采用（风格近似）
RESTful	REST-style	符合 REST 约束的接口设计风格	采用，按资源路径组织
HTTP	HyperText Transfer Protocol	超文本传输协议	采用
HTTPS	HTTP Secure	基于 TLS 的 HTTP	生产部署时采用
URL	Uniform Resource Locator	统一资源定位符	采用
URI	Uniform Resource Identifier	统一资源标识符	概念引用
JSON	JavaScript Object Notation	JavaScript 对象表示法，接口数据格式	采用
CORS	Cross-Origin Resource Sharing	跨域资源共享	采用，`CORSMiddleware`
CRUD	Create Read Update Delete	创建、读取、更新、删除	采用（单词本、记录）
DTO	Data Transfer Object	数据传输对象，本项目对应 Pydantic 请求模型	采用
SPA	Single Page Application	单页应用，前端路由不刷新整页	采用（React Router）
SSG	Static Site Generation	静态站点生成	未采用
SSR	Server-Side Rendering	服务端渲染	未采用
SSE	Server-Sent Events	服务器推送事件，常用于流式输出	**未采用**，当前对话为一次性 JSON 响应
WebSocket	WebSocket Protocol	全双工通信协议	**未采用**
JWT	JSON Web Token	JSON 网络令牌，常用于无状态鉴权	**未采用**，系统无登录
SDK	Software Development Kit	软件开发工具包	采用（OpenAI Python SDK）

缩写 / 术语	英文全称	中文含义	本项目采用情况
NPM	Node Package Manager	Node.js 包管理器	采用（前端依赖）
TS	TypeScript	微软推出的 JavaScript 强类型超集	采用
TSX	TypeScript XML	TypeScript 中的 JSX 语法文件扩展名	采用
JSX	JavaScript XML	React 的声明式 UI 语法扩展	间接采用（经 TSX）
CSS	Cascading Style Sheets	层叠样式表	采用（Tailwind CSS 生成）
DOM	Document Object Model	文档对象模型	采用（浏览器运行时）
HTML	HyperText Markup Language	超文本标记语言	采用

1.3.3 领域智能体术语

缩写 / 术语	英文全称	中文含义	本项目采用情况
AI	Artificial Intelligence	人工智能	采用（领域背景）
LLM	Large Language Model	大语言模型，本项目指 DeepSeek `deepseek-chat`	采用（生成层）
Agent	Intelligent Agent	智能体，能感知输入、调用工具并产生动作的软件实体	采用（领域智能体）
Domain Agent	Domain-Specific Agent	领域智能体，面向特定领域知识与任务约束的智能体	采用，本项目核心定位
ReAct	Reasoning and Acting	"推理—行动"交替的智能体范式，模型自行决定调用哪个工具	**未采用。**本项目使用规则式意图路由，工具由 `if` 关键字匹配选择，不由模型自主规划
RAG	Retrieval-Augmented Generation	检索增强生成，通常指向量检索后注入提示词	**未采用。**本项目使用 SQL 结构化检索（词库、真题切片），不构建向量索引，不把检索结果整体拼接进模型上下文作为唯一事实来源
Tool Calling	Function Calling / Tool Use	模型输出结构化工具调用请求的机制	**未采用模型驱动的工具调用。**工具由后端编排层直接调用
Tool Trace	Tool Execution Trace	工具执行轨迹，记录工具名、参数、摘要与耗时	采用，字段 `tool_trace`
Citation	Citation	引用/出处，本项目指真题句子切片出处	采用，字段 `citations`
Prompt	Prompt	提示词，输入给大模型的指令文本	采用
Prompt Engineering	Prompt Engineering	提示词工程，通过结构化指令约束模型输出	采用

缩写 / 术语	英文全称	中文含义	本项目采用情况
System Prompt	System Prompt	系统提示词，定义模型角色与硬约束	采用，`server/prompts.py`
Hallucination	Hallucination	幻觉，模型生成与事实不符的内容	采用（设计要规避的问题）
Token	Token	模型计费与上下文计量单位	采用（输出上限约束）
Temperature	Temperature	采样温度，越低输出越稳定	采用，取值 0.15–0.2
Grounding	Grounding	事实落地，把回答约束到可核查证据	采用（工具结果与引用）

1.3.4 数据与存储术语

缩写 / 术语	英文全称	中文含义	本项目采用情况
DB	Database	数据库	采用
DBMS	Database Management System	数据库管理系统	采用（SQLite）
SQLite	SQLite	嵌入式关系型数据库，单文件、零运维	采用，`server/data/app.db`
SQL	Structured Query Language	结构化查询语言	采用（原生 SQL）
ORM	Object-Relational Mapping	对象关系映射，把表映射为编程语言对象	**未采用**。项目直接使用 Python 标准库 `sqlite3` 与手写 SQL，`db.py` 仅定义 schema 与连接
WAL	Write-Ahead Logging	预写式日志，SQLite 的并发读写模式	采用，`PRAGMA journal_mode = WAL`
FK	Foreign Key	外键	部分采用，见 5.7 与数据库设计说明书
PK	Primary Key	主键	采用
DDL	Data Definition Language	数据定义语言	采用（`CREATE TABLE`）
DML	Data Manipulation Language	数据操纵语言	采用（`INSERT/UPDATE/DELETE/SELECT`）
JSONB	JSON Binary	PostgreSQL 的二进制 JSON 类型	未采用

缩写 / 术语	英文全称	中文含义	本项目采用情况
SM-2	SuperMemo 2	一种间隔重复记忆算法	**部分采用。**项目保留 `easiness`、`interval_days`、`repetition` 字段，但当前复习只做"记住则间隔翻倍、忘记回到 1 天"，未实现完整的质量评分与 easiness 更新
CRUD	Create Read Update Delete	增删改查	采用
UUID	Universally Unique Identifier	通用唯一识别码，本项目用于生成 `auto_` / `custom_` 篇章 ID	采用（`uuid.uuid4().hex`）

1.3.5 框架与平台术语

缩写 / 术语	英文全称	中文含义	本项目采用情况
Python	Python	解释型通用编程语言	采用（后端）
FastAPI	FastAPI	基于 Python 类型注解的高性能 Web 框架	采用
Pydantic	Pydantic	Python 数据校验与序列化库，FastAPI 的请求模型基础	采用（v2）
Uvicorn	Uvicorn	ASGI 服务器，用于运行 FastAPI	采用
ASGI	Asynchronous Server Gateway Interface	异步服务器网关接口	采用（FastAPI 底层）
WSGI	Web Server Gateway Interface	同步 Web 服务器网关接口	未采用
React	React	声明式前端 UI 库	采用（v19）
Vite	Vite	前端构建与开发服务器工具	采用（v8）
Tailwind CSS	Tailwind CSS	原子化 CSS 框架	采用（v4）
DeepSeek	DeepSeek	深度求索公司的大模型服务，兼容 OpenAI 接口	采用（可选）
OpenAI	OpenAI	大模型服务商，其接口规范成为行业兼容标准	采用（兼容协议）
dotenv	python-dotenv	读取 `.env` 环境变量的库	采用
httpx	HTTPX	Python 的 HTTP 客户端库	采用（PDF 下载）
pypdf	pypdf	Python 的 PDF 文本抽取库	采用
pytest	pytest	Python 测试框架	间接采用（`server/tests` 使用 `unittest`，未强制 pytest）

缩写 / 术语	英文全称	中文含义	本项目采用情况
CI	Continuous Integration	持续集成	**未采用**
CD	Continuous Deployment	持续部署	**未采用**
Tauri	Tauri	桌面应用框架	未采用
Electron	Electron	桌面应用框架	未采用

1.3.6 文档与图表术语

缩写 / 术语	英文全称	中文含义	本项目采用情况
Mermaid	Mermaid	以文本描述生成图表的工具	采用（设计图集）
PlantUML	PlantUML	以文本描述生成 UML 图的工具	采用（用例图、通信图）
PNG	Portable Network Graphics	便携式网络图形格式	采用（图表导出）
SVG	Scalable Vector Graphics	可缩放矢量图形	采用（图表导出）

1.4 参考资料

编号	名称	版本 / 位置	用途
REF-01	GB/T 8567-2006《计算机软件文档编制规范》	国家标准	本文档章节结构与表述依据
REF-02	GB/T 9385-2008《计算机软件需求规格说明规范》	国家标准	需求条目组织参考
REF-03	项目仓库 README	`README.md`，代码基线 `0688e14`	项目形态、启动方式、边界说明
REF-04	前后端分离架构说明	`docs/ARCHITECTURE.md`	运行拓扑、四个工具、三类工作流、API 概览
REF-05	软件工程答辩项目档案	`docs/PROJECT-DOSSIER.md`	设计取舍、质量验证、技术债
REF-06	考研英语智学 Agent 需求文档	`docs/需求文档.md` v0.2	功能需求 FR-00～FR-10、非功能需求 NFR-01～NFR-09
REF-07	本地启动与配置教程	`docs/LOCAL-SETUP.md`	环境变量、端口、Windows 启动细节
REF-08	已导入真题清单	`docs/IMPORTED-EXAMS.md`	题库覆盖年份与规模
REF-09	题库建库与发布操作说明	`docs/题库建库与发布操作说明.md`	草稿、审核、发布、撤回流程
REF-10	概要设计架构与 UML 图集	`docs/概要设计-架构与UML图集.md`	本文档引用的图 1～图 7

编号	名称	版本 / 位置	用途
REF-11	FastAPI 官方文档	`https://fastapi.tiangolo.com/`	路由、依赖注入、Pydantic 用法
REF-12	React 官方文档	`https://react.dev/`	组件与状态管理
REF-13	Mermaid 官方文档	`https://mermaid.js.org/`	图表语法
REF-14	SQLite 官方文档	`https://www.sqlite.org/docs.html`	WAL、外键、事务
REF-15	公共考研词库 KyleBing/english-vocabulary	`https://github.com/KyleBing/english-vocabulary`，BSD-3-Clause	词库来源与许可证
REF-16	公开真题 PDF 仓库 Fantasia1999/kaoyanzhenti	`https://github.com/Fantasia1999/kaoyanzhenti`	真题来源（不随仓库分发 PDF）
REF-17	DeepSeek 开放平台文档	`https://api-docs.deepseek.com/`	OpenAI 兼容接口参数
REF-18	后端运行说明	`server/README.md`	模块职责、数据表、题库入口
REF-19	前端运行说明	`web/README.md`	前端脚本与目录

2 系统总体设计

2.1 需求规定回顾

2.1.1 产品定位与目标

系统面向考研英语一/英语二考生，把阅读训练拆成四个可验证的动作：看懂句子、做对题目、记住词汇、形成反馈。核心产品主张为：**工具负责事实，模型负责表达，数据库负责状态。**

需求文档给出的五项产品目标与本设计的对应关系如下：

目标编号	需求目标	设计实现	验证点
G-01	让句子可拆	五步拆解 + 结构树 + `sentence_breakdowns` 缓存	任意已发布句子可通过 `/api/sentences/{id}/breakdown` 获取结构化拆解
G-02	让选项可验	作答态/解析态分离 + A–D 逐项陷阱 + 定位句高亮	提交前不显示答案与陷阱标签，提交后展示四选项归因
G-03	让事实可查	`tool_trace` 与 `citations` 由后端生成，不经过模型	前端 `ToolTrace` 组件展示工具名、摘要、耗时与出处

目标编号	需求目标	设计实现	验证点
G-04	让学习有状态	单词本、练习记录、复习时间写入 SQLite	一次加词或答题后，首页指标与报告同步变化
G-05	让素材可扩展	题库快照恢复 + 命令行 JSON 建库 + 草稿审核发布	新增篇章后阅读、长难句、对话三个入口均可选择

2.1.2 功能需求清单

编号	功能模块	优先级	设计对应章节
FR-00	全局框架与导航	P0	4.1、5.1
FR-01	首页仪表盘	P0	5.1
FR-02	长难句拆解	P0	3.3、3.4、5.3
FR-03	阅读真题训练与选项分析	P0	3.3、3.4、5.4
FR-04	AI 对话	P0	3.1、3.2、3.4、5.2
FR-05	我的单词本与间隔复习	P0	3.3、5.5
FR-06	资料库（词库与真题篇章）	P1	5.6、5.7
FR-07	学习报告	P1	5.8
FR-08	领域知识库	P1	5.6
FR-09	题库建设与发布（团队侧）	P0	5.7
FR-10	LLM 能力与降级	P0	3.2、6.1

2.1.3 非功能需求清单

编号	类别	需求指标	设计措施
NFR-01	性能	本地数据库查询秒级返回；未缓存拆解首次 5–10 秒，需加载提示	索引按主键与唯一约束；拆解结果缓存；前端 loading 状态
NFR-02	可用性	后端未启动时前端不白屏	`AppStore.refresh` 失败置 `connected=false`；页面使用独立 error 态；`localStorage` 离线兜底
NFR-03	兼容性	Chrome / Edge 近两个大版本；桌面优先，窄屏折叠导航	响应式 `lg` 断点；`Layout` 抽屉模式
NFR-04	运行环境	Python 3.11/3.12；Node.js 20+/24+	`requirements.txt` 与 `package.json` 版本约束
NFR-05	安全	API Key 只存服务端 `.env`，不进入 `VITE_*`	`.gitignore` 忽略 `.env`、`*.db`、`*.pdf`
NFR-06	可追溯	工具调用与真题出处必须来自后端工具层	模型只接收工具结果，不能生成 `tool_trace` / `citations`

编号	类别	需求指标	设计措施
NFR-07	数据完整性	建库校验发布数量、外键与完整性	`setup_team.py` 校验 68 篇/340 题；`question_bank.readiness`
NFR-08	可维护性	前端 API 调用统一收口；领域文案集中	`web/src/lib/api.ts` 唯一客户端；`web/src/data/content.ts`
NFR-09	可部署性	前后端两个独立进程	Vite 代理开发；`VITE_API_BASE` 指向生产 API

2.1.4 明确不做的事项

需求文档已明确以下非目标，设计不得隐性扩张：

1. 不做整卷模考、写作、翻译、完形等非阅读题型。
2. 不做社交、排行榜、课程售卖。
3. 不做公网多用户 SaaS、登录鉴权与数据隔离。
4. 不做原生手机 App 与微信小程序，本期仅 Web 响应式。
5. 不宣称覆盖 2010—2026 全量真题，不宣称收录官方 5,500 词全集。
6. 不做商业题库批量抓取；PDF 由使用者自行提供并确认授权。

2.2 运行环境

2.2.1 软硬件与依赖版本

类别	项目	版本 / 配置	说明
操作系统	Windows 11 / macOS / Linux	开发验证以 Windows 11 为主	`LOCAL-SETUP.md` 给出 Windows 细节
后端语言	Python	3.11 或 3.12	`docs/需求文档.md` NFR-04
Web 框架	FastAPI	`fastapi>=0.115.0`	`server/requirements.txt`
ASGI 服务器	Uvicorn	`uvicorn[standard]>=0.30.0`	启动 `uvicorn app:app --port 8000`
数据校验	Pydantic	`pydantic>=2.0.0`	请求模型与字段约束
LLM SDK	OpenAI Python SDK	`openai>=1.0.0`	通过 `base_url` 指向 DeepSeek
环境变量	python-dotenv	`python-dotenv>=1.0.0`	读取 `server/.env`
PDF 解析	pypdf	`pypdf>=4.0.0`	文本型 PDF 抽取
HTTP 客户端	httpx	`httpx>=0.27.0`	下载公开 PDF
后端端口	TCP	`8000`	`http://127.0.0.1:8000`
前端语言	TypeScript	`^7.0.2`	`web/package.json`

类别	项目	版本 / 配置	说明
前端框架	React / React DOM	`^19.3.1`	函数组件 + Hooks
前端路由	React Router DOM	`^7.18.4`	8 路由 + 404 重定向
构建工具	Vite	`^8.3.5`	开发与生产构建
样式	Tailwind CSS / @tailwindcss/vite	`^4.3.3`	原子化样式
图标	lucide-react	`^1.61.0`	线性图标
前端端口	TCP	`5173`	`http://localhost:5173`
数据库	SQLite	Python 标准库 `sqlite3` 内置版本	单文件 `server/data/app.db`
数据库模式	WAL + 外键	`journal_mode=WAL`、`foreign_keys=ON`	`db.py` schema 首部
浏览器	Chrome / Edge	近两个大版本	桌面优先，窄屏可折叠
运行时网络	可选外网	访问 DeepSeek API	无 Key 时离线可用

2.2.2 环境变量与配置文件

变量名	作用域	是否必填	示例	说明
`LLM_API_KEY`	后端	否	`sk-...`	未配置时 `is_configured()` 为假，系统走确定性路径
`LLM_BASE_URL`	后端	否	`https://api.deepseek.com/v1`	OpenAI 兼容地址
`LLM_MODEL`	后端	否	`deepseek-chat`	模型标识
`VITE_API_BASE`	前端构建期	否	`/api` 或 `https://api.example.com/api`	未设置时默认 `/api`，由 Vite 代理转发

配置加载规则：

1. 后端通过 `python-dotenv` 读取 `server/.env`；`.env` 已被 `.gitignore` 忽略，仓库只保留 `.env.example` 模板。
2. 前端变量以 `VITE_` 前缀暴露，构建时内联，因此**严禁**把 `LLM_API_KEY` 写入任何 `VITE_*` 变量。
3. 开发环境 `web/vite.config.ts` 将 `/api` 代理到 `http://127.0.0.1:8000`。
4. 生产环境可将前端静态文件部署到任意静态服务器，用 `VITE_API_BASE` 指向独立后端。

2.2.3 启动与初始化

后端首次启动时，`FastAPI` 的 `lifespan` 回调执行 `init_db()`：

1. 创建 `server/data/app.db`（不存在时）。

2. 执行 `SCHEMA` 中的建表语句（`CREATE TABLE IF NOT EXISTS`）。
3. 执行 `migrate_schema()` 幂等补充新增字段。
4. 执行 `seed_database()` 导入词库、示例文章、陷阱规则与演示状态。

前端启动时，`AppStoreProvider` 立即调用 `api.getState()` 同步单词本与练习记录；同时各页面按需请求自己需要的数据，避免首屏加载全库。

2.3 基本设计概念与分层处理流程

2.3.1 核心设计概念

概念一：工具负责事实，模型负责表达，数据库负责状态。 系统把"什么算事实"和"怎么表述"彻底分开。大纲属性、真题出处、正确答案、单词本状态、练习记录全部来自 SQLite 或确定性规则；大模型只被允许改写教学讲解的措辞。

概念二：证据敏感路径锁定确定性。 长难句、选项分析、单词本归档三类请求不进入模型改写。原因是测试中出现过"2023 上下文混入 2024 句子"的串篇问题，因此把证据敏感路径锁死，宁可表达生硬也不允许事实漂移。

概念三：一次操作必须改变下一次页面看到的数据。 加词写入 `user_vocab_book`，答题写入 `practice_records`，复习更新 `next_review`，这些状态会被首页、单词本、报告页读取，形成"训练—记录—报告—复习"的闭环，而不是一次性聊天。

概念四：可插拔生成层。 模型是可选依赖。没有 Key、网络失败、JSON 非法时，系统仍能给出完整的确定性解析与工具链，只是少了教学化的措辞润色。

2.3.2 四层结构与处理流程

系统按图 2（见《概要设计-架构与UML图集》）划分为四层，另有一个外部生成层：

层次	组成	职责	不允许做的事
表现层	`web/src/`：8 个页面、11 个组件、`AppStore`、`api.ts`	页面渲染、交互状态、请求编排、工具链可视化	不直接持有领域数据，不直接拼 SQL，不直连模型
业务逻辑与智能体编排层	`server/app.py`、`server/workflows.py`、`server/prompts.py`	路由与校验、意图识别、工具编排、结构化回答组装、运行日志	不直接执行领域 SQL，不生成工具链与引用
工具与领域知识层	`server/tools.py`、`server/seed.py`、`server/exam_import.py`、`server/question_bank.py`	四个确定性工具、词库与真题检索、导入校验、题库状态机	不调用模型改写事实，不渲染页面
数据持久层	`server/db.py`、`server/data/*`	schema、连接、迁移、词库与题库文件	不含业务判断

层次	组成	职责	不允许做的事
外部生成层	`server/llm.py` + DeepSeek API	开放讲解、导入结构化、句子五步拆解	不生成 `tool_trace`、`citations`、正确答案、数据库写入结果

2.3.3 一次典型请求的处理流程

以"学生提交一道阅读题"为例，端到端流程如下（对应图 6 与图 7 的对象协作）：

```
1. 学生在 OptionsPage 选择 B 并点击提交
2. OptionsPage.handleSubmit 先在本地镜像一条记录（乐观更新，避免刷新前报告不动）
3. api.runOptions(questionId, "B") → POST /api/workflow/options
4. app.py 反序列化 OptionsRequest，Depends(get_db) 注入 SQLite 连接
5. workflows.run_options_workflow：
   5.1 get_question(conn, question_id) 读取题干、四选项、analysis_json、steps_json
   5.2 get_sentence(conn, source_sentence_id) 读取定位句
   5.3 search_past_exams(conn, query=定位句前 70 字, limit=3) 生成 citations
   5.4 record_option_attempt(conn, question, "B") 写入 practice_records 并返回陷阱类型
   5.5 _log_run("options", ...) 写入 agent_runs
6. 返回 question / source_sentence / citations / is_correct / selected_trap / record / tool_trace
7. 前端进入解析态：逐项陷阱、定位句高亮、工具链
8. 报告页读取 AppStore.records 或后端 /api/state，正确率与薄弱题型同步变化
```

该流程说明了三个关键设计点：

- **乐观更新与最终一致**：前端先本地镜像，后端返回后以服务端记录为准；后端失败时页面提示"只保存在本地"。
- **证据链完整**：`citations` 来自 `search_past_exams`，`tool_trace` 来自工具执行，均不经模型。
- **状态闭环**：`practice_records` 是唯一事实来源，前端状态只是缓存。

2.4 系统结构

2.4.1 总体功能模块结构

系统总体功能模块结构见图 1（《概要设计-架构与UML图集》）；参与者与用例边界见图 3。按功能域划分为四个一级子系统：

```
考研英语领域智能体助学系统
├── 学生端 Web 工作台子系统
│   ├── 学情仪表盘 DashboardPage
│   ├── Agent 智能对话 ChatPage
│   ├── 长难句精析 SentencePage
│   ├── 阅读选项拆解 OptionsPage
│   ├── 生词本与熟词僻义 WordbookPage + KnowledgePage
│   ├── 真题切片库 LibraryPage
│   └── 学情诊断报告 ReportPage
├── 服务接入子系统
│   ├── FastAPI 路由与请求校验 app.py
│   └── 数据库连接注入 get_db
├── 领域智能体子系统
│   ├── 编排 workflows.py
│   ├── 工具 tools.py
│   ├── 提示词 prompts.py
│   └── 生成层 llm.py
├── 数据与题库管理子系统
│   ├── schema 与连接 db.py
│   ├── 种子导入 seed.py
│   ├── 导入校验 exam_import.py
│   ├── 题库状态机 question_bank.py
│   └── 快照脚本 scripts/
└── 早期 MVP 子系统（历史保留）
    └── kaoyan_agent Streamlit
```

2.4.2 子系统划分与边界

子系统	对应目录	主要制品	对外边界
学生端 Web 工作台	`web/src/`	8 页面、11 组件、`api.ts`、`AppStore`	仅通过 `/api` 与后端通信
服务接入	`server/app.py`	26 个 `/api` 路由、请求模型	对前端暴露 HTTP JSON
领域智能体	`server/workflows.py`、`tools.py`、`llm.py`、`prompts.py`	3 条工作流、4 个工具、5 类生成函数	对 app.py 暴露 Python 函数
数据与题库管理	`server/db.py`、`seed.py`、`exam_import.py`、`question_bank.py`、`scripts/`	10 张表、校验器、状态机、快照脚本	对工具层暴露 SQL 与校验函数；对维护者暴露 CLI
早期 MVP	`kaoyan_agent/`	Streamlit 单体应用	独立运行，不参与主链路

2.4.3 前后端职责边界

关注点	前端职责	后端职责
页面与交互	全部	不涉及
领域数据	不持有，按需请求	SQLite 唯一来源

关注点	前端职责	后端职责
工具调用	不直接调用	`workflows.py` 编排 `tools.py`
模型调用	不直接调用，不持有 Key	`llm.py` 调用 DeepSeek
工具链展示	渲染 `tool_trace`	生成 `tool_trace`
学习状态	乐观更新与本地兜底	持久化与最终一致
题库管理	无入口	命令行脚本

2.5 人工处理过程与尚未解决的问题

2.5.1 人工处理过程

系统中以下环节**必须由人完成**，不能由程序自动替代：

编号	人工环节	执行者	触发时机	操作说明
M-01	LLM API Key 配置	系统管理员	首次部署或更换模型	在 `server/.env` 填写 `LLM_API_KEY` / `LLM_BASE_URL` / `LLM_MODEL`
M-02	题库 JSON 草稿导入	内容维护者	新增真题	`python scripts/question_bank.py` 导入 JSON，生成 `draft` 状态篇章
M-03	解析生成或补齐	内容维护者	草稿缺少选项分析	调用 `question_bank.generate_analysis`，模型生成后仍为 `pending`
M-04	人工审核	内容维护者	发布前	`transition(action="review")`，校验解析、证据、元数据完整；记录 `reviewed_by` / `reviewed_at`
M-05	发布或开放基础练习	内容维护者	审核通过后	`transition(action="publish")` 或 `publish-practice`
M-06	答案核对	内容维护者	导入含 `inferred` 答案时	对照原卷核对 `correct_answer` 与 `answer_source_note`
M-07	撤回与更正	内容维护者	发现错误	`transition(action="withdraw")` 回到草稿，修订后重新审核发布
M-08	数据库备份与快照导出	系统管理员	版本交付或换机	复制 `server/data/app.db` 或执行 `python scripts/export_demo.py`
M-09	快照恢复	系统管理员	新环境部署	`python scripts/setup_team.py`，已有数据库时拒绝覆盖

2.5.2 尚未解决的问题

以下问题在代码中真实存在，设计文档必须如实披露，不得用"已解决"表述掩盖：

编号	问题	影响	当前状态	后续方向
P-01	无登录与用户隔离	所有学习记录共用一份 SQLite，不能公网多用户部署	未实现	增加 `user_id`、鉴权与数据隔离
P-02	无自动化测试与 CI	重构风险高，回归依赖手工	`server/tests` 有 3 个 `unittest` 文件，但无 CI	接入 pytest + 前端 Vitest + CI
P-03	无数据库迁移工具	schema 变更依赖 `CREATE TABLE IF NOT EXISTS` 与 `migrate_schema`	已实现轻量幂等迁移，无版本号	引入迁移脚本版本表
P-04	导入 URL 缺 SSRF 防护	`_download_pdf_text` 会请求用户提供的 URL	HTTP 导入接口已返回 410，但代码仍在	域名白名单、内网地址拦截、大小限制
P-05	对话未写 `agent_runs`	对话问题排查能力弱	`_log_run` 只在句子/选项工作流调用	对话工作流补日志与请求 ID
P-06	重复导入无数据库唯一约束	同一 Text 可能产生重复篇章	应用层 `_auto_duplicate` / `_manual_duplicate` 查重	增加 `(year, exam_type, text_number)` 唯一索引（需先清理历史数据）
P-07	答案正确性依赖公开资料	无官方答案校验	人工核对，`answer_source` 标记来源	引入官方答案比对与抽检
P-08	扫描版 PDF 无 OCR	只能处理文本型 PDF	未实现	接入 OCR 服务
P-09	复习算法为简化 SM-2	未使用质量评分与 easiness 更新	间隔按 1→2→4→8→16→30 天倍增	实现完整 SM-2
P-10	报告存在前后端两套统计	前端本地聚合，后端 `/api/report` 另有实现	页面使用前端聚合	统一到后端聚合
P-11	CORS 允许任意来源	`allow_origins=["*"]`	开发期方便	生产环境收敛白名单
P-12	无请求 ID 与结构化日志	排查困难	仅 `logging.warning`	增加请求 ID、统一错误模型
P-13	早期 Streamlit 与主系统并存	两套实现增加维护成本	`kaoyan_agent/` 保留可运行	明确归档或下线

3 领域智能体（Domain Agent）专项设计

3.1 智能体与业务系统集成架构

3.1.1 设计目标：打破"纯聊天壳子"

传统"聊天壳子"的问题是：对话结束后什么都不改变，用户拿到的是一段无法追溯的文本。本系统的集成目标是让对话成为**业务系统的一个入口**，与结构化页面共享同一套工具、同一份数据库与同一套状态。

具体表现为四个"共享"：

共享对象	对话入口 `ChatPage`	结构化入口	共享机制
篇章与句子	通过 `context_id` 选择已发布篇章，回答引用该篇句子	`SentencePage` 通过 `/api/passages` 选择同一批篇章	两者都只读 `publication_status='published'` 的 `exam_passages` / `exam_sentences`
工具链	对话回答带 `tool_trace`	长难句页、选项页同样带 `tool_trace`	三条工作流共用 `tools.py` 的四个工具与统一 trace 结构
单词本状态	对话说"把 X 加入单词本"直接写库	`SentencePage` 点"加入单词本"写同一张表	共用 `add_to_vocab_book` 与 `user_vocab_book`
练习记录	对话问"分析这道错题"可读到记录	`OptionsPage` 提交写入 `practice_records`	共用 `record_option_attempt` 与 `practice_records`

3.1.2 双向联动的真实链路

方向一：结构化页面 → 对话（状态被对话读取）

1. `OptionsPage` 提交答案 → `POST /api/workflow/options` → `record_option_attempt` 写 `practice_records`。
2. 用户在 `ChatPage` 提问"安排本周复习计划" → 意图命中复习类 → `_run_chat_deterministic` 调用 `get_report(conn)` 与到期词查询。
3. `get_report` 读取同一张 `practice_records`，按题型聚合正确率、筛选 `accuracy < 0.7` 的薄弱类型，读取 `next_review <= now` 的到期词，生成建议。
4. 对话回答中的"你有 N 个单词到期""某题型正确率 X%"来自刚写入的记录，不是模型编造。

方向二：对话 → 结构化页面（状态被结构化页面读取）

1. 用户在对话中说"把 inescapable 加入单词本" → 意图命中归档类 → `check_syllabus_vocab` 识别候选词 → `add_to_vocab_book` 写 `user_vocab_book`。
2. 前端收到 `tool_trace` 后调用 `AppStore.refresh()` 重新拉取 `/api/state`。
3. `WordbookPage` 立刻出现该词，`DashboardPage` 的"单词本"指标 +1，到期横幅在次日显示。
4. 该词的下次复习由 `review_word` 继续推进，与对话入口无关，保证状态一致。

方向三：对话内部复用结构化工作流（不是重新实现一遍）

`_run_chat_deterministic` 在意图命中长难句时，直接调用 `run_sentence_workflow(conn, sentence_id)`；命中选项时直接调用 `run_options_workflow`。因此对话里的句子解析与 `SentencePage` 的解析来自同一段代码、同一套引用过滤规则、同一份缓存。

3.1.3 联动边界与限制

必须如实说明当前实现的边界：

1. `ChatPage` **没有页面跳转链接**（未使用 `useNavigate` 或 `Link` 跳转到 `/sentence` 或 `/options`）。联动发生在数据层，不是路由层。
2. 对话的 `context_id` 只影响本次请求的检索范围；切换上下文不会改变其他页面的当前选择。
3. 对话回答中的句子是"当前篇章最长句"（`max(context_sentences, key=len)`），不是用户逐句指定的句子；用户若要精确拆解某句，应使用 `SentencePage`。
4. 单词本归档只归档第一个命中的大纲词（`candidates[0]`），一次对话只加一个词。

3.1.4 集成架构图引用

集成关系见图 2（分层架构图）与图 7（通信图）。图 2 中的 `ChatPage → api.ts → app.py → workflows.py → tools.py → db.py` 与 `SentencePage/OptionsPage → 同一 tools.py` 两条路径汇合于工具层，是"对话与结构化功能同源"的结构证据。

3.2 提示词工程体系

3.2.1 提示词资产清单

资产	位置	作用	调用函数
`SYSTEM_PROMPT`	`server/prompts.py`	定义"研语智学（YanYu Agent）"的领域教研人格、七条硬准则与输出风格	所有 LLM 调用共用
对话润色提示词	`server/llm.py:polish_chat`	把确定性结果改写为 3–5 个分节的教学讲解	`polish_chat`
单题分析提示词	`server/llm.py:analyze_imported_question`	为一道导入题生成题型、定位、四选项陷阱与步骤	`analyze_imported_question`
整篇抽取提示词	`server/llm.py:extract_exam_text`	从原始文本抽取 passage + 5 题 + 选项	`extract_exam_text`
批量分析提示词	`server/llm.py:analyze_imported_questions_batch`	一次调用分析整篇 5 题的选项陷阱	`analyze_imported_questions_batch`
五步拆解提示词	`server/llm.py:analyze_sentence_breakdown`	生成主干、从句、修饰、翻译层、语法点、考点、结构树	`analyze_sentence_breakdown`

3.2.2 系统提示词的角色约束

`SYSTEM_PROMPT` 将模型设定为"深耕中国硕士研究生入学统一考试英语（一/二）命题规律的首席教研智能体"，其约束可归纳为七条：

编号	约束原文要点	设计意图
SP-1	解析词汇或句子前，优先使用 `check_syllabus_vocab` 的结果，不臆造大纲属性、考频或熟词僻义	把"是否超纲、考频多少"交给 SQLite，模型不得凭记忆回答
SP-2	引用例句必须使用 `search_past_exams` 的返回结果，并标注年份、试卷、Text、段、句；无匹配时说明"仿真示例"	杜绝编造真题出处
SP-3	发现核心词或典型陷阱时，可调用 `add_to_vocab_book` / `record_trap_mistake` 写入学习档案，并在回答末尾说明已归档	让模型理解"动作"的存在，但实际调用由后端执行
SP-4	长难句严格按"找主干 → 剥离从句 → 划分修饰 → 组合精译 → 语法点与生词"五步输出	固定领域方法论，避免自由发挥
SP-5	选项分析严格按"审题定位 → 同义替换链 → 干扰项定性 → 排除 → 确认答案"输出	固定选项分析范式
SP-6	陷阱优先归入偷换概念、扩大/缩小范围、正反混淆、无中生有、过度推理、张冠李戴	与 `trap_rules` 八类规则对齐，保证标签可统计
SP-7	若前端请求 JSON，只输出符合 Schema 的 JSON，不添加寒暄	保证程序可解析

风格约束为"严谨、干练、结构化，像一位面对面辅导的考研名师"。

3.2.3 输出格式控制

项目采用三层格式控制，逐层收紧：

第一层：系统提示词层。 `SYSTEM_PROMPT` 第 7 条要求"只输出 JSON，不添加寒暄"。

第二层：用户提示词层。 每个函数在 user prompt 中给出**带字段名的 JSON 模板**，并逐条列出字段取值约束。例如 `analyze_sentence_breakdown` 的模板要求：

```
{
  "is_long": true,
  "main": "主句原文",
  "mainNote": "用中文说明主干与核心谓语",
  "clauses": [{"text": "原文中的从句片段", "label": "定语从句", "note": "修饰谁"}],
  "modifiers": [{"text": "原文中的修饰片段", "label": "介词短语", "note": "修饰对象"}],
  "translation": "整句自然中文翻译",
  "translationLayers": ["按语义层次的中文翻译"],
  "grammar": [{"name": "语法点名称", "note": "在本句中的体现"}],
  "examPoints": ["命题人可能设置的考点"],
  "tree": [{"role": "主句", "label": "主句原文", "children": []}]
}
```

第三层：程序校验层（最关键）。 模型输出后不直接入库，而是由 Python 逐字段校验、截断与兜底：

校验点	函数	规则
JSON 提取	`_extract_json`	去除 Markdown 代码围栏，只取第一个 JSON 对象
结构清洗	`_sanitize`	校验 `intro` 为字符串、`sections` 为列表、单节字段类型正确
题型白名单	`VALID_QUESTION_TYPES`	不在五类题型内一律降级为"细节题"
陷阱白名单	`VALID_TRAPS`	不在八类陷阱内一律降级为"待分析"
正确答案强制	`analyze_imported_question`	正确答案的 `trapType` 强制为"正确"，其他选项若模型标"正确"则改为"待分析"并提示复核
长度截断	各分析函数	`explanation[:800]`、`trapType[:20]`、`steps[:6]`
原文一致性	`analyze_sentence_breakdown`	提示词要求 `main` 必须是原句连续片段；程序在解析失败时回退整句
英文来源约束	`analyze_sentence_breakdown`	提示词要求所有英文必须来自原句，不得补充原句之外的英文

采样参数：`temperature` 取 0.15–0.2，`max_tokens` 按任务规模取 1400–6000，优先请求 `response_format={"type":"json_object"}`；若网关不支持该参数，捕获异常后去掉 `response_format` 重试一次。

3.2.4 考研英语命题规律注入

命题规律通过三种方式注入，而不是让模型自由发挥：

1. **方法论模板注入**：五步拆解法与选项五步法直接写入提示词，模型只能按固定步骤组织内容。
2. **陷阱分类体系注入**：八类陷阱名称与定义来自 `trap_rules` 表与 `VALID_TRAPS` 集合，模型输出必须落在该闭集内，保证前端标签可统计、可着色。
3. **证据优先注入**：要求先定位原文、再给结论；`source_quote` 必须逐字摘录，不能确定时留空，主旨题允许留空待人工复核。对导入题目，若 `source_quote` 在原文中不唯一，`locate_quote` 不会强行绑定句子，而是标记 `location_status=unverified`。

3.2.5 提示词版本与可维护性

当前提示词以 Python 字符串常量形式存在，无独立版本号。设计上建议后续将提示词外置为模板文件并增加版本字段，但**当前实现没有这样做**，文档不虚构该能力。

3.3 领域工具集设计

3.3.1 工具设计总则

设计项	规则
统一返回	每个工具返回 `(业务结果, trace)` 二元组，trace 字段固定为 `tool`、`args`、`summary`、`duration_ms`
单一职责	一个工具只做一件事：查词、查真题、写单词本、写练习记录
幂等优先	加词前先查重，已存在则返回原记录并说明"未重复写入"
只读已发布	`search_past_exams`、`get_sentence`、`get_question` 默认过滤 `publication_status='published'`
不调用模型	四个工具全部是确定性 SQL 与规则，不含 LLM 调用
失败可解释	空输入、未命中、未提交答案都返回有意义的 summary，而不是抛异常

3.3.2 工具一：`check_syllabus_vocab`（大纲词与熟词僻义核验）

功能：把一段英文文本或一个词表转换为"是否在大纲、等级、基础释义、熟词僻义、切片内出现频次"的结构化词条列表。

入参 Schema：

参数	类型	必填	约束	说明
`conn`	`sqlite3.Connection`	是	由依赖注入提供	数据库连接
`text_or_words`	`str | list[str]`	是	字符串按词切分；列表逐项切分	待核验文本或词表

文本模式的处理链：

1. `extract_words` 用正则提取英文词，转小写，去掉首尾连字符与撇号。
2. 过滤长度小于 3 的词、停用词（`STOPWORDS`）与纯数字。
3. 去重并保持出现顺序，最多取前 80 个词。
4. 对每个词调用 `_find_vocab`，按 `_lemma_candidates` 生成的词形候选依次查 `syllabus_vocabulary.word`。
5. 未命中返回 `in_syllabus=false`、`level="超纲"`、`exam_frequency=0`。

词形还原规则（`_lemma_candidates`）覆盖：所有格 `'s`、`ies→y`、`ied→y`、`es` 去尾、`s` 去尾、`ed` 去尾与双写辅音、`ing` 去尾与补 `e`、`ly` 去尾、`er` 去尾、`est` 去尾。这是规则式还原，不是词典级词形分析，因此存在少量漏配。

返回结构：

```
{
  "items": [
    {
      "word": "inescapable",
      "matched": "inescapable",
      "in_syllabus": true,
      "level": "核心",
      "basic_meaning": "不可避免的",
      "rare_meaning": null,
      "exam_frequency": 2,
      "source": "本地大纲词库"
    },
    {
      "word": "anthropogenic",
      "matched": null,
      "in_syllabus": false,
      "level": "超纲",
      "basic_meaning": "",
      "rare_meaning": null,
      "exam_frequency": 0,
      "source": "本地大纲词库"
    }
  ],
  "tool_trace": {
    "tool": "check_syllabus_vocab",
    "args": {"word_count": 19, "preview": ["the", "trajectory"]},
    "summary": "检索 19 个实词，命中 16 个大纲词，其中 2 个含熟词僻义",
    "duration_ms": 1
  }
}
```


> 说明：HTTP 接口 `POST /api/tools/check-vocab` 把二元组拆成 `{"items": ..., "tool_trace": ...}`；工作流内部则直接使用二元组。

触发时机： 1. 句子工作流每次执行时对整句核验。 2. 对话意图为"熟词僻义/核心词/生词/词汇"时。 3. 对话意图为"加入单词本"时，先核验以确定候选词。 4. 前端工具直连调试接口。

容错机制： - 空文本返回空列表与"检索 0 个实词"摘要，不抛异常。 - 词库未命中不是错误，返回超纲标记。 - 最多 80 词，避免长文本造成慢查询。 - 词形还原失败时退化为原词查询。

3.3.3 工具二：`search_past_exams`（真题切片检索）

功能：按原文片段或语法标签检索已发布真题句子切片，强制附带出处。

入参 Schema：

参数	类型	必填	约束	说明
`conn`	`sqlite3.Connection`	是	—	数据库连接

参数	类型	必填	约束	说明
`query`	`str`	否	默认空串，最多取前 80 字符参与 LIKE	原文片段
`grammar`	`str | list[str] | None`	否	字符串按 `,，;； |/\s` 切分，最多 4 个标签	语法标签
`limit`	`int`	否	默认 5，内部限制 1–20	返回条数

检索逻辑：

```
SELECT s.*
FROM exam_sentences AS s
JOIN exam_passages AS p ON p.id = s.passage_id
WHERE p.publication_status = 'published'
  AND (lower(s.text) LIKE ? OR lower(s.source_label) LIKE ?)
  AND (s.grammar_tags LIKE ? OR ...)
ORDER BY s.year DESC, s.passage_id, s.paragraph, s.sentence_index
LIMIT ?
```

返回结构：

```
{
  "items": [
    {
      "id": "auto_xxx_s12",
      "source": "自动导入 · 2024 英语一 Text 1 · 第2段第3句",
      "passage_id": "auto_xxx",
      "year": 2024,
      "exam_type": "英语一",
      "text": "The trajectory remains concerning...",
      "grammar_tags": ["定语从句", "分词短语"]
    }
  ],
  "tool_trace": {
    "tool": "search_past_exams",
    "args": {"query": "The trajectory remains", "grammar": ["定语从句"], "limit": 4},
    "summary": "检索到 4 条真题切片，优先引用 自动导入 · 2024 英语一 Text 1 · 第2段第3句",
    "duration_ms": 2
  }
}
```

触发时机： 1. 句子工作流按 `grammar_tags[:2]` 检索同类结构。 2. 选项工作流按定位句前 70 字检索出处。 3. 对话的引用展示。 4. 工具直连 `/api/tools/search-exams`。

容错机制： - 无匹配返回空数组，summary 为"检索到 0 条真题切片"。 - `query` 最长 80 字符，避免超长 LIKE。 - `limit` 强制收敛到 1–20。 - 仅查已发布篇章，草稿不会泄露到学生端。

引用约束（在编排层实现）：句子工作流会把当前句强制置顶 `citations`，并过滤为同 `passage_id` 最多 3 条，避免不同年份句子串篇。该过滤不在工具内部完成，而在 `run_sentence_workflow` 中完成。

3.3.4 工具三：`add_to_vocab_book`（生词语境归档）

功能：把一个单词连同出处与原句语境写入 `user_vocab_book`，并安排首次复习。

入参 Schema：

参数	类型	必填	约束	说明
`conn`	`sqlite3.Connection`	是	—	数据库连接
`word`	`str`	是	取第一个英文词并转小写	待归档单词
`context`	`str`	否	入库截断为 500 字符	原句语境
`source`	`str`	否	默认"Agent 自动归档"	出处标签

处理逻辑：

1. 用 `WORD_RE` 提取第一个英文词并转小写；为空则返回空字典与"单词为空，未写入"。
2. 查 `user_vocab_book.spelling` 是否已存在；存在则直接返回原记录，summary 为"已在单词本中，未重复写入"。
3. 查 `syllabus_vocabulary` 取基础释义；若该词有 `rare_meaning`，释义拼接为"基础释义；熟词僻义：……"。
4. 未收录时释义写"（词库未收录，请手动补充释义）"，`is_outline=0`。
5. 写入 `next_review = now + 1 天`，`familiarity=0`、`review_count=0`、`easiness=2.5`、`interval_days=1`、`repetition=0`。
6. 提交事务并回读记录。

返回结构：


```
{
  "word": {
    "id": "w7",
    "spelling": "inescapable",
    "phonetic": "",
    "meaning": "不可避免的",
    "source": "长难句拆解 · 2024 英语一 Text 1",
    "context": "The trajectory remains concerning...",
    "isOutline": true,
    "familiarity": 0,
    "reviewCount": 0,
    "nextReview": "2026-11-10T21:00:00"
  },
  "tool_trace": {
    "tool": "add_to_vocab_book",
    "args": {"word": "inescapable", "source": "长难句拆解 · 2024 英语一 Text 1"},
    "summary": "已将 inescapable 写入单词本，首次复习安排在明天",
    "duration_ms": 3
  }
}
```

触发时机： 1. 长难句页"加入单词本"按钮。 2. 对话中"加入/记到/归档/收藏 + 单词本"意图。 3. 资料库词库列表行内加词。 4. `POST /api/wordbook`。

> 注意：`POST /api/wordbook` 的请求模型包含 `meaning` 字段，但 `add_to_vocab_book` 实际以词库释义为准，不使用客户端传入的 `meaning`。这是当前实现的真实行为，文档不作美化。

容错机制： - 空词不写入。 - 重复词幂等返回。 - 词库未收录仍可入库，标记 `is_outline=0`。 - 语境截断到 500 字符，避免超长文本撑大数据库。

3.3.5 工具四：`record_trap_mistake`（错题与陷阱归档）

实现函数：`record_option_attempt(conn, question, user_answer)`。工具链中显示名固定为 `record_trap_mistake`。

功能：把一次作答写入 `practice_records`，记录正误与所选选项的陷阱类型。

入参 Schema：

参数	类型	必填	约束	说明
`conn`	`sqlite3.Connection`	是	—	数据库连接
`question`	`dict`	是	由 `get_question` 产出，含 `id`、`number`、`question_type`、`correct_answer`、`analysis`	题目对象
`user_answer`	`str | None`	否	`None` 表示未提交，只返回 trace 不写库	用户答案

处理逻辑：

1. 若 `user_answer` 为空，返回 `(None, trace)`，summary 为"未提交答案，跳过错题归档"。
2. `is_correct = user_answer == question["correct_answer"]`。
3. 正确时 `trap_type="正确"`；错误时取 `question["analysis"][user_answer]["trapType"]`，缺失则为"未归类"。
4. 写入 `practice_records(question_id, question_type, user_answer, correct_answer, is_correct, trap_type, agent_feedback)`，`agent_feedback` 保存该选项分析的 JSON。
5. 返回前端记录对象，`id` 格式为 `r{lastrowid}`。

返回结构：

```
{
  "record": {
    "id": "r22",
    "questionId": "auto_xxx_q21",
    "questionType": "细节题",
    "userAnswer": "B",
    "correctAnswer": "C",
    "isCorrect": false,
    "trapType": "偷换概念",
    "time": "2026-11-09T21:10:00"
  },
  "tool_trace": {
    "tool": "record_trap_mistake",
    "args": {"question_id": "auto_xxx_q21", "trap_type": "偷换概念"},
    "summary": "已记录第 21 题作答：B，命中陷阱「偷换概念」",
    "duration_ms": 2
  }
}
```


触发时机： 1. 选项工作流在 `user_answer` 非空时调用。 2. `POST /api/records` 通用记录接口。

容错机制： - 未提交答案不写库，不产生脏记录。 - 陷阱类型缺失降级为"未归类"。 - 错误选项分析与正确答案冲突时导入期已修正为"待分析"，避免错误标签污染统计。

3.3.6 辅助查询与聚合函数

除四个核心工具外，`tools.py` 还提供以下函数，它们不产生 `tool_trace`，但被工作流与接口直接使用：

函数	职责	关键返回
`get_sentence`	读取句子并校验篇章已发布	`id`、`text`、`grammar_tags`、`source`、`char_start/end`
`get_question`	读取题目与四选项分析	`options`、`analysis`、`steps`、`correct_answer`、`location_status`
`list_sentences`	列出已发布句子	`items[]`
`list_questions`	列出已发布题目摘要	`items[]`
`get_state`	同步单词本与练习记录	`words[]`、`records[]`
`review_word`	复习反馈与间隔推进	更新后的 word
`remove_word`	删除单词	`bool`
`get_report`	学情聚合	`total`、`accuracy`、`by_type`、`weak_types`、`due_words`、`suggestions`
`get_meta`	规模指标	9 个统计字段
`get_trap_rules`	八条陷阱规则	`name`、`definition`、`signals`、`example`、`tone`

函数	职责	关键返回
`get_rare_words`	高频熟词僻义样例	6 条
`search_vocab`	关键词检索	最多 `limit` 条
`list_vocab`	分页浏览与筛选	`items`、`total`

3.4 多步任务工作流与轨迹追踪

3.4.1 三条工作流

工作流	入口函数	输入	步骤	输出
句子工作流	`run_sentence_workflow`	`sentence_id`	`get_sentence` → `check_syllabus_vocab` → `search_past_exams` → 引用约束 → `_log_run`	`sentence`、`vocabulary`、`vocabulary_all_count`、`citations`、`tool_trace`
选项工作流	`run_options_workflow`	`question_id`、`user_answer`	`get_question` → `get_sentence` → `search_past_exams` → `record_option_attempt` → `_log_run`	`question`、`source_sentence`、`citations`、`is_correct`、`selected_trap`、`record`、`tool_trace`
对话工作流	`run_chat_workflow`	`message`、`context_id`	`_run_chat_deterministic` 意图路由 → 工具/子工作流 → 可选 `polish_chat`	`intro`、`sections`、`tool_trace`、`citations`、`llm_used`

3.4.2 对话意图路由表

`_run_chat_deterministic` 采用关键字匹配，按下表顺序判断，命中即返回：

优先级	意图	触发关键字	处理	是否走 LLM
1	单词本归档	含"单词本"且含"加入/记到/归档/收藏"	`check_syllabus_vocab` + `add_to_vocab_book`	否（状态变更永不走模型）
2	长难句	"从句/长难句/拆解/句子"	`run_sentence_workflow` + `_get_sentence_breakdown`	否（证据锁定）
3	选项	"选项/陷阱/错误选项/排除"	`run_options_workflow`	否（证据锁定）
4	熟词僻义	"熟词僻义"等	词库检索 + 八类陷阱知识	是（开放讲解）
5	核心词	"核心词/关键词/生词/词汇"	当前篇章词库筛选	否
6	复习计划	"计划/复习/单词"	`get_report` + 到期词	是（开放讲解）
7	段落主旨	"主旨/总结/段落/态度"	原文首尾与题干定位抽取	否

优先级	意图	触发关键字	处理	是否走 LLM
8	默认	其他	能力与数据状态说明	是（可关闭）

3.4.3 轨迹数据结构

每个工具的 trace 结构固定：

```
{
  "tool": "check_syllabus_vocab",
  "args": {"word_count": 19, "preview": ["the", "trajectory", "of"]},
  "summary": "检索 19 个实词，命中 16 个大纲词，其中 2 个含熟词僻义",
  "duration_ms": 1
}
```

字段语义：

字段	类型	说明
`tool`	string	工具名，与 `tools.py` 中的 `_trace` 第一个参数一致
`args`	object	本次调用的关键参数摘要，不是完整原始输入，避免泄露长文本
`summary`	string	面向用户的中文执行摘要，前端直接展示
`duration_ms`	number	工具内部耗时，最小为 1 毫秒

3.4.4 运行日志 `agent_runs`

`_log_run` 把一次工作流执行写入 `agent_runs`：

字段	内容
`workflow`	`sentence` 或 `options`
`input_json`	工作流输入，例如 `{"sentence_id": "..."}`
`output_json`	输出摘要，例如 `{"sentence_id": "...", "vocab_count": 12, "citation_count": 4}`
`tool_trace_json`	完整工具链
`created_at`	SQLite 默认时间

当前**对话工作流不写** `agent_runs`，这是已知技术债 P-05。

3.4.5 前端 `ToolTrace.tsx` 的配合机制

`ToolTrace` 是一个纯展示组件，接收 `tools: ToolCall[]` 与可选 `title`：

1. 若 `tools` 为空，组件不渲染。

2. 每个工具显示图标、`tool` 名称、`summary` 文本与 `duration_ms`。
3. 工具名可展开 `args`，用于答辩演示时展示"调用了什么参数"。
4. 样式使用统一设计系统的卡片、Tag 与 `scrollbar-thin`。

配合关系可概括为：**后端生成轨迹，前端只消费轨迹。**前端没有任何计算工具链的逻辑，也没有在模型回答中解析工具调用的逻辑，因此不会出现"模型说调用了某工具但实际没调用"的展示层幻觉。

3.4.6 工作流与图集的对应

图	对应内容
图 4a	对话与长难句诊断的活动流程，含意图路由、工具调用、缓存判断、模型分支
图 4b	真题导入与切片入库的活动流程
图 6	一次长难句提问的时序，含 3 次数据库读取、缓存分支、模型降级
图 7	对象协作拓扑与带序号消息

---

4 接口设计

4.1 用户接口

4.1.1 导航结构

前端采用左侧固定侧栏 + 右侧内容区的经典工作台布局，导航项与路由严格对应：

序号	导航项	路由	页面组件	图标语义
1	首页	`/`	`DashboardPage`	仪表盘
2	长难句拆解	`/sentence`	`SentencePage`	扫描文本
3	阅读真题训练	`/options`	`OptionsPage`	靶心
4	AI 对话	`/chat`	`ChatPage`	对话气泡
5	我的单词本	`/wordbook`	`WordbookPage`	书签
6	资料库	`/library`	`LibraryPage`	层叠资料
7	学习报告	`/report`	`ReportPage`	柱状图
8	领域知识库	`/knowledge`	`KnowledgePage`	图书馆

未知路由 `*` 重定向到 `/`。页面标题随路由切换，由 `Layout` 中的 `ScrollToTop` 组件设置 `document.title`。

4.1.2 视觉与交互规范

规范项	规定
画布背景	Tailwind `slate-50`（`#F8FAFC`）
卡片	白色背景、`border-slate-200`、轻阴影、12px 圆角
深色 Banner	`slate-800`（`#1E293B`）、16px 圆角
按钮与 Tag	8px 圆角
功能色	蓝（句子拆解）、绿（正确/定制）、青（知识）、黄（复习）、紫（AI/练习）、红（错误/对照）
图标	lucide-react，线性，1.5–2px 描边
布局	240px 固定侧栏 + 自适应主区；指标行宽屏 4 列
卡片强调	4px 左色条；悬浮 `-translate-y-0.5` + `shadow-md`
字体	`Inter` / `Noto Sans SC` / `PingFang SC` / `Microsoft YaHei`，通过 `index.css` 的 `@theme` 定义
焦点可见	`:focus-visible` 2px 描边，偏移 2px
滚动条	`scrollbar-thin` 工具类，6px 宽
响应式	`lg` 以上双栏/四列；窄屏单列，导航折叠为抽屉
提示	全局 Toast，三种语义：success / info / error
空状态	统一使用 `EmptyState` 组件，给出下一步动作
加载状态	骨架、淡入动画 `animate-toast-in`、输入中三点动画 `typing-dot`

4.1.3 页面级交互规范

页面	关键交互	状态约束
仪表盘	四张指标卡可点击跳转；到期横幅"去复习"；今日任务三项	后端不可用时显示"本地模式"
长难句	文章下拉 → 句子下拉级联；默认选最长句；五步 Tab 切换	加载中、读取失败、无句子、未发布篇章均有独立空态
阅读训练	年份/试卷/Text 联动筛选；左文右题独立滚动；作答后提交	未提交不显示答案与陷阱；基础练习只保存作答
AI 对话	上下文下拉；6 个快捷胶囊；Enter 发送、Shift+Enter 换行	后端不可用提示检查 FastAPI，不使用假答案
单词本	全部/今日复习/添加三页签；先回忆后显示释义	先乐观更新，再落库；失败保留本地
资料库	词库搜索 300ms 防抖；等级筛选；排序；分页 50/页	已加入单词本的词禁用重复添加
学习报告	正确率、五类题型、薄弱项、7 天柱状图	无记录时引导去做第一题
知识库	题型/陷阱/方法关键词检索	实时显示命中条目数

4.1.4 键盘与可访问性

当前已实现：Enter 发送、Shift+Enter 换行、Tab 原生焦点顺序、`:focus-visible` 描边、按钮 `disabled` 状态禁止点击、`aria-label`（部分下拉框）。

尚未实现：完整 ARIA 角色标注、屏幕阅读器全链路验证、键盘快捷键体系。该差距已在 P-01 ~ P-13 之外单独列为后续无障碍改进项，本文档不宣称已通过无障碍评测。

4.2 外部接口

4.2.1 与 LLM 服务的通信协议

系统通过 OpenAI Python SDK 调用 DeepSeek，协议为 **OpenAI 兼容的 Chat Completions**。

项目	配置
协议	HTTPS + JSON
鉴权	`Authorization: Bearer <LLM_API_KEY>`，由 SDK 自动添加
基础地址	`LLM_BASE_URL`，示例 `https://api.deepseek.com/v1`
端点	`POST {LLM_BASE_URL}/chat/completions`
模型	`LLM_MODEL`，示例 `deepseek-chat`
请求头	`Content-Type: application/json`
超时	50s（对话润色、单题分析）、90s（整篇抽取）、120s（批量分析、句子拆解）
重试	仅当网关不支持 `response_format` 时去掉该参数重试一次；不做网络层重试
流式输出	未启用，一次性返回完整 JSON

4.2.2 请求体

```
{
  "model": "deepseek-chat",
  "messages": [
    {"role": "system", "content": "你是研语智学（YanYu Agent）……"},
    {"role": "user", "content": "请基于下面的工具结果，把回答改写成一节面向考研学生的结构化讲解……"}
  ],
  "temperature": 0.2,
  "max_tokens": 1600,
  "response_format": {"type": "json_object"}
}
```

字段说明：

字段	类型	取值	设计意图
`model`	string	`deepseek-chat`	统一模型标识

字段	类型	取值	设计意图
`messages[0]`	object	`role=system`	注入领域人格与七条硬约束
`messages[1]`	object	`role=user`	任务指令 + 工具上下文 + JSON 模板
`temperature`	number	0.15–0.2	降低随机性，保证解析稳定
`max_tokens`	number	1400–6000	按任务规模控制成本与截断风险
`response_format`	object	`{"type":"json_object"}`	要求返回 JSON；不支持时降级重试

4.2.3 响应体

```
{
  "id": "chatcmpl-xxx",
  "object": "chat.completion",
  "created": 1762700000,
  "model": "deepseek-chat",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "{\"intro\":\"...\",\"sections\":[{\"title\":\"结构\",\"body\":\"...\"}]}"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {"prompt_tokens": 1200, "completion_tokens": 800, "total_tokens": 2000}
}
```

程序只读取 `choices[0].message.content`，然后：

1. `_extract_json` 去掉 Markdown 围栏并提取 JSON。
2. `_sanitize` 或各业务函数逐字段校验。
3. 校验失败返回 `None`，由调用方回退确定性结果。

4.2.4 外部接口异常分类

异常	检测方式	处理
未配置 Key	`is_configured()` 为假	不发起请求，直接返回 `None`，`llm_used=false`
网络超时	SDK 抛异常	捕获并 `logger.warning`，返回 `None`
401/403	HTTP 状态异常	捕获异常并记录 `logger.warning`，返回 `None`，回退确定性结果
429 限流	HTTP 429	当前**没有自动退避重试**，直接回退；这是已知限制
5xx	HTTP 5xx	捕获异常并记录 `logger.warning`，返回 `None`，回退确定性结果

异常	检测方式	处理
返回非 JSON	`_extract_json` 返回 `None`	回退
JSON 字段缺失	各函数的类型校验	缺字段使用默认值或回退
陷阱标签越界	`VALID_TRAPS` 白名单	改为"待分析"
题型越界	`VALID_QUESTION_TYPES` 白名单	改为"细节题"

4.3 内部接口

4.3.1 接口总览

`server/app.py` 共暴露 26 个 `/api` 路由与 1 个根路由。完整清单如下：

编号	方法	路径	请求体 / 参数	成功响应要点	主要错误码
API-00	GET	`/`	无	服务名、版本、docs、health 地址	—
API-01	GET	`/api/health`	无	`status`、`database`、9 项规模指标、`llm_configured`、`llm_model`	500
API-02	GET	`/api/meta`	无	`vocab_count`、`sentence_count`、`question_count`、`passage_count`、`trap_count`、`wordbook_count`、`record_count`、`database`、`knowledge_version`	500
API-03	GET	`/api/state`	无	`{words: Word[], records: StudyRecord[]}`	500
API-04	GET	`/api/sentences`	无	`{items: [{id, source, grammar_tags, text}]}`	500
API-05	GET	`/api/passages`	无	`{items: PassageSummary[]}`，按年份倒序去重	500
API-06	GET	`/api/passages/{passage_id}/sentences`	路径参数	`{passage, items: [{id, index, paragraph, text, source, word_count, is_long}]}`	404

编号	方法	路径	请求体 / 参数	成功响应要点	主要错误码
API-07	GET	`/api/sentences/{sentence_id}/breakdown`	路径参数	`SentenceBreakdown`（含 `llmUsed`）	404
API-08	GET	`/api/questions`	无	`{items: [{id, year, exam_type, text_number, number, question_type, content, correct_answer}]}`	500
API-09	GET	`/api/knowledge`	无	`{traps: [], rare_words: [], meta: {}}`	500
API-10	POST	`/api/workflow/sentence`	`{"sentence_id": "..."}`	`{sentence, vocabulary, vocabulary_all_count, citations, tool_trace}`	404、422
API-11	POST	`/api/workflow/options`	`{"question_id": "...", "user_answer": "B"}`	`{question, source_sentence, citations, is_correct, selected_trap, record, tool_trace}`	404、422
API-12	POST	`/api/workflow/chat`	`{"message": "...", "context_id": "..."}`	`{intro, sections, tool_trace, citations, llm_used}`	422、500
API-13	GET	`/api/wordbook`	无	`{items: Word[]}`	500
API-14	POST	`/api/wordbook`	`{"spelling", "meaning", "source?", "context?"}`	`{ok, word, tool_trace}`	422、500
API-15	DELETE	`/api/wordbook/{spelling}`	路径参数	`{ok: true, spelling}`	404
API-16	POST	`/api/wordbook/review`	`{"spelling": "...", "remembered": true}`	`{ok: true, word}`	404、422
API-17	POST	`/api/records`	`{"question_id", "question_type", "user_answer", "correct_answer"}`	`{ok: true, record}`	404、422
API-18	GET	`/api/report`	无	`{total, correct, accuracy, by_type, trap_distribution, weak_types, due_words, suggestions}`	500
API-19	GET	`/api/questions/full`	无	`{items: FullQuestion[]}`	500

编号	方法	路径	请求体 / 参数	成功响应要点	主要错误码
API-20	POST	`/api/exams/import`	旧请求模型	**410**	410
API-21	POST	`/api/exams/auto-import`	旧请求模型	**410**	410
API-22	GET	`/api/agent/runs`	`?limit=10`	**410**	410
API-23	GET	`/api/vocab/search`	`?q=&limit=20`	`{items: VocabRow[]}`	422
API-24	GET	`/api/vocab`	`?q=&level=&sort=freq&offset=0&limit=50`	`{items: VocabRow[], total}`	422
API-25	POST	`/api/tools/check-vocab`	`{"text": "..."}` 或 `{"words": ["..."]}`	`{items, tool_trace}`	400、422
API-26	GET	`/api/tools/search-exams`	`?q=&grammar=&limit=5`	`{items, tool_trace}`	422

4.3.2 核心接口响应示例

API-01 `GET /api/health`

```
{
  "status": "ok",
  "database": "SQLite",
  "vocab_count": 5064,
  "sentence_count": 1295,
  "question_count": 340,
  "passage_count": 68,
  "trap_count": 8,
  "wordbook_count": 7,
  "record_count": 20,
  "knowledge_version": "2026.10",
  "llm_configured": true,
  "llm_model": "deepseek-chat"
}
```

> 说明：`health` 先写 `status` 与 `database:"connected"`，随后用 `get_meta()` 展开，`meta` 中的 `database:"SQLite"` 会覆盖前者，因此实际返回 `"SQLite"`。以上数值为快照口径，不同初始化方式（仅

> seed / setup_team）会有差异，前端只展示不硬编码。

API-10 `POST /api/workflow/sentence`

请求：

```
{"sentence_id": "auto_xxx_s12"}
```

响应：

```
{
  "sentence": {
    "id": "auto_xxx_s12",
    "passage_id": "auto_xxx",
    "year": 2024,
    "exam_type": "英语一",
    "text_number": 1,
    "paragraph": 2,
    "sentence_index": 3,
    "text": "The trajectory remains concerning...",
    "grammar_tags": ["定语从句", "分词短语"],
    "source": "自动导入 · 2024 英语一 Text 1 · 第2段第3句",
    "char_start": 812,
    "char_end": 980
  },
  "vocabulary": [
    {
      "word": "trajectory",
      "matched": "trajectory",
      "in_syllabus": true,
      "level": "核心",
      "basic_meaning": "轨迹；趋势",
      "rare_meaning": null,
      "exam_frequency": 3,
      "source": "本地大纲词库"
    }
  ],
  "vocabulary_all_count": 16,
  "citations": [
    {
      "id": "auto_xxx_s12",
      "source": "自动导入 · 2024 英语一 Text 1 · 第2段第3句",
      "year": 2024,
      "exam_type": "英语一",
      "text": "The trajectory remains concerning...",
      "grammar_tags": ["定语从句", "分词短语"]
    }
  ],
  "tool_trace": [
    {
      "tool": "check_syllabus_vocab",
      "args": {"word_count": 19, "preview": ["the", "trajectory"]},
      "summary": "检索 19 个实词，命中 16 个大纲词",
      "duration_ms": 1
    },
    {
      "tool": "search_past_exams",
      "args": {"query": "", "grammar": ["定语从句", "分词短语"], "limit": 4},
      "summary": "定位到当前篇 3 条真题切片，优先引用 自动导入 · 2024 英语一 Text 1 · 第2段第3句",
      "duration_ms": 2
    }
```

```
  ]
}
```

API-12 `POST /api/workflow/chat`

请求：

```
{"message": "解析这个句子的从句", "context_id": "auto_xxx"}
```

响应：


```
{
  "intro": "这句话的主干是 The trajectory remains concerning，后面由 that 引导同位语从句解释轨迹内容。",
  "sections": [
    {
      "title": "句子结构",
      "code": "The trajectory remains concerning...",
      "body": "主句为系表结构，that 从句作同位语。"
    },
    {
      "title": "考点",
      "items": ["同位语从句与定语从句的区分", "remain 系动词用法"]
    }
  ],
  "tool_trace": [
    {"tool": "check_syllabus_vocab", "args": {"word_count": 19}, "summary": "检索 19 个实词，命中 16 个大纲词", "duration_ms": 1},
    {"tool": "search_past_exams", "args": {"grammar": ["同位语从句"], "limit": 4}, "summary": "定位到当前篇 3 条真题切片", "duration_ms": 2}
  ],
  "citations": [
    {"id": "auto_xxx_s12", "source": "自动导入 · 2024 英语一 Text 1 · 第2段第3句", "year": 2024, "exam_type": "英语一", "text": "...", "grammar_tags": ["同位语从句"]}
  ],
  "llm_used": false
}
```


API-14 `POST /api/wordbook`

请求：

```
{
  "spelling": "inescapable",
  "meaning": "不可避免的",
  "source": "长难句拆解 · 2024 英语一 Text 1",
  "context": "The trajectory remains concerning..."
}
```

响应：

```
{
  "ok": true,
  "word": {
    "id": "w8",
    "spelling": "inescapable",
    "phonetic": "",
    "meaning": "不可避免的",
    "source": "长难句拆解 · 2024 英语一 Text 1",
    "context": "The trajectory remains concerning...",
    "isOutline": true,
    "familiarity": 0,
    "reviewCount": 0,
    "nextReview": "2026-11-10T21:00:00"
  },
  "tool_trace": {
    "tool": "add_to_vocab_book",
    "args": {"word": "inescapable", "source": "长难句拆解 · 2024 英语一 Text 1"},
    "summary": "已将 inescapable 写入单词本，首次复习安排在明天",
    "duration_ms": 3
  }
}
```


API-16 `POST /api/wordbook/review`

请求：


```
{"spelling": "inescapable", "remembered": true}
```


响应：


```
{
  "ok": true,
  "word": {
    "id": "w8",
    "spelling": "inescapable",
    "meaning": "不可避免的",
    "source": "长难句拆解 · 2024 英语一 Text 1",
    "context": "...",
    "isOutline": true,
    "familiarity": 0.15,
    "reviewCount": 1,
    "nextReview": "2026-11-11T21:00:00"
  }
}
```


> 复习算法：记住时 `familiarity +0.15`、`interval_days = min(30, interval_days × 2)`、`repetition +1`；忘记时 `familiarity −0.1`、`interval_days = 1`、`repetition = 0`。前端本地常量 `REVIEW_INTERVALS = [1,2,4,8,16,30]` 仅用于即时展示，最终以后端返回为准。

API-19 `GET /api/questions/full`

单项结构：

```
{
  "id": "auto_xxx_q21",
  "number": 21,
  "questionType": "细节题",
  "content": "According to Paragraph 2, ...",
  "options": {"A": "...", "B": "...", "C": "...", "D": "..."},
  "correctAnswer": "C",
  "analysis": {
    "A": {"trapType": "偷换概念", "explanation": "原文说局部，选项扩大为全部。"},
    "B": {"trapType": "正反混淆", "explanation": "原文态度为担忧，选项说充分。"},
    "C": {"trapType": "正确", "explanation": "与原文同义替换。"},
    "D": {"trapType": "无中生有", "explanation": "原文未提成本。"}
  },
  "steps": ["定位第2段", "比对选项", "排除绝对项"],
  "location": "第2段第3句",
  "sourceSentence": "...",
  "sourceSentenceId": "auto_xxx_s12",
  "answerSource": "provided",
  "answerSourceNote": "用户提供答案序列，尚未核对权威来源",
  "reviewStatus": "approved",
  "locationStatus": "exact_quote",
  "sourceQuote": "...",
  "sourceHighlights": [],
  "passage": "全文……",
  "passageId": "auto_xxx",
  "year": 2024,
  "examType": "英语一",
  "textNumber": 1
}
```

4.3.3 请求模型与字段约束

模型	字段	约束
`ChatRequest`	`message`	1–4000 字符
	`context_id`	0–80 字符，默认空串
`SentenceRequest`	`sentence_id`	1–80 字符
`OptionsRequest`	`question_id`	1–80 字符
	`user_answer`	可空，最多 4 字符
`WordCreateRequest`	`spelling`	1–80 字符
	`meaning`	1–500 字符

模型	字段	约束
	`source`	0–200 字符
	`context`	0–1000 字符
`ReviewRequest`	`spelling`	1–80 字符
	`remembered`	布尔
`RecordRequest`	四字段	字符串；`user_answer` 服务端校验必须是 A/B/C/D
`VocabCheckRequest`	`text` / `words`	二者至少一个
`AutoImportRequest`	`year` / `exam_type` / `text_number` / `answer_key`	年份 2000–2100；试卷限"英语一/英语二"；Text 1–4；答案序列最多 200 字符
`ExamImportRequest`	`passage` / `content` / `options` / `correct_answer`	文章 50–20000 字符；题干 5–2000 字符；四选项齐全；答案 A–D

4.3.4 错误响应格式

FastAPI 默认错误：

```
{"detail": "sentence not found"}
```

校验错误（422）：

```
{
  "detail": [
    {"loc": ["body", "message"], "msg": "String should have at least 1 character", "type":
"string_too_short"}
  ]
}
```

导入校验错误（422，自定义结构）：

```
{
  "detail": {
    "code": "invalid_exam",
    "message": "真题校验失败，未入库",
    "issues": ["答案序列须恰有 5 个 A/B/C/D，实际 4 个"]
  }
}
```

重复导入（409，自定义结构）：

```
{
  "detail": {
    "code": "duplicate_exam",
    "message": "该真题已导入，请先核对现有题库；本次未写入或覆盖。"
  }
}
```

停用接口（410）：

```
{"detail": "题库管理已迁移到团队离线命令行 scripts/question_bank.py，学生接口不接受导入或管理操作"}
```

4.3.5 接口分页、排序与筛选约定

参数	接口	默认	约束	说明
`q`	`/api/vocab`、`/api/vocab/search`	空	最多 80 字符	匹配单词、基础释义、熟词僻义
`level`	`/api/vocab`	空	最多 16 字符	基础 / 核心 / 拔高
`sort`	`/api/vocab`	`freq`	`freq` 或 `alpha`	频次倒序或字母序
`offset`	`/api/vocab`	0	≥ 0	分页偏移
`limit`	`/api/vocab`	50	1–200	每页条数
`limit`	`/api/vocab/search`	20	1–100	搜索条数
`limit`	`/api/tools/search-exams`	5	1–20	切片条数
`limit`	`/api/agent/runs`	10	1–50	但路由已 410

5 模块详细层级设计

本章按"模块职责 → 输入项 → 输出项 → 核心算法/处理逻辑 → 关联源码文件"统一结构描述 8 个业务模块。模块顺序与左侧导航一致。

5.1 学情仪表盘模块

5.1.1 模块职责

在首屏 5 秒内回答"我现在有什么、该做什么"：展示真题篇章、大纲词汇、单词本、已练习四项规模指标；提示到期复习；提供四个核心功能入口；展示今日学习任务与最近 5 条练习。

5.1.2 输入项

输入	来源	说明
规模指标	`GET /api/meta`	9 个字段，页面使用 `passage_count`、`vocab_count`、`wordbook_count`、`record_count`、`question_count`、`sentence_count`、`trap_count`
单词本与记录	`AppStore`（启动时 `/api/state`）	`words`、`records`、`dueWords`、`connected`
本地缓存	`localStorage["kaoyan-studio-v1"]`	后端不可用时的离线兜底

5.1.3 输出项

输出	形式
四张指标卡	数值 + 单位 + 跳转链接
到期词横幅	"今天有 N 个单词到期" + "去复习"按钮
四张功能卡	长难句、阅读训练、AI 对话、单词本
今日任务	复习到期词、拆 1 句、分析 1 题
最近练习	最近 5 条：题型、作答、正误、相对时间

5.1.4 核心算法与处理逻辑

1. 首次挂载时并行执行：`api.getMeta()` 与 `AppStore` 的 `refresh()`。
2. `getMeta` 成功则渲染真实规模，失败则展示本地 seed 数据并标记后端不可用。
3. `dueWords` 由 `AppStore` 的 `useMemo` 计算：`new Date(word.nextReview).getTime() <= Date.now()`，并按到期时间升序。
4. 最近练习取 `records.slice(0, 5)`，`time` 字段经前端格式化为"几分钟前/几小时前"。
5. 点击指标卡使用 `Link` 跳转；点击"去复习"跳转 `/wordbook` 并切到"今日复习"页签。
6. 后端失败时 `connected=false`，页面显示"本地模式"，但不阻塞浏览。

对应图 1 的功能入口、图 7 的 `StudyReportGenerator` 读取路径。

5.1.5 关联源码文件

文件	作用
`web/src/pages/DashboardPage.tsx`	页面组件、指标卡与最近练习
`web/src/components/StatCard.tsx`	指标卡
`web/src/components/FeatureCard.tsx`	功能入口卡
`web/src/components/PageHero.tsx`	页面头部
`web/src/store/AppStore.tsx`	`words`、`records`、`dueWords`、`connected`

文件	作用
`web/src/lib/api.ts`	`getMeta`
`server/app.py`	`/api/meta`
`server/tools.py`	`get_meta`、`get_state`

5.2 Agent 智能对话模块

5.2.1 模块职责

提供"有数据库的辅导老师"式对话：可切换已发布篇章作为上下文，回答长难句、选项、核心词、熟词僻义、复习计划、段落主旨等问题，并在回答中展示工具链与真题出处，同时可执行"加入单词本"等状态变更动作。

5.2.2 输入项

输入	来源	说明
用户消息	输入框或快捷胶囊	1–4000 字符
上下文篇章	`GET /api/passages` 下拉	`context_id`，可为空
篇章句子/题目数	`/api/passages` 返回的计数	顶部状态栏展示
后端连接状态	`AppStore.connected`	顶部状态标识

5.2.3 输出项

输出	形式
结构化回答	`intro` + 3–5 个 `sections`（可含 `body`、`items`、`code`）
工具链	`tool_trace`，交给 `ToolTrace` 渲染
引用	最多 3 条 `citations` 标签
生成来源	`llm_used=true` 显示"DeepSeek 生成"，否则显示确定性标识
状态变更提示	归档成功提示与单词本刷新

5.2.4 核心算法与处理逻辑

1. 发送时先追加用户消息并置输入框 disabled，防止重复提交。
2. `api.chat(text, contextId)` → `POST /api/workflow/chat`。
3. 后端 `_run_chat_deterministic` 按 3.4.2 的优先级做关键字路由。
4. 长难句分支选择 `context_sentences` 中字符最长的句子，调用 `run_sentence_workflow`，再查 `sentence_breakdowns` 缓存；未命中调用 `analyze_sentence_breakdown` 并写缓存。
5. 选项分支调用 `run_options_workflow`，若用户未提交答案则不写记录。

6. 归档分支先 `check_syllabus_vocab(message)` 找候选词，取第一个大纲词写库。
7. 开放讲解分支在 `is_configured()` 为真时调用 `polish_chat`，失败保留确定性结果。
8. 前端收到响应后追加助手消息；若 `tool_trace` 中存在 `add_to_vocab_book`，调用 `refresh()` 同步单词本。
9. 清空对话恢复初始欢迎语，不删除后端记录。

5.2.5 关联源码文件

文件	作用
`web/src/pages/ChatPage.tsx`	对话界面、上下文选择、快捷问题、消息渲染
`web/src/components/ToolTrace.tsx`	工具链渲染
`web/src/lib/api.ts`	`chat`、`getPassages`
`server/app.py`	`/api/workflow/chat`
`server/workflows.py`	`run_chat_workflow`、`_run_chat_deterministic`、`_context`、`_get_sentence_breakdown`
`server/llm.py`	`polish_chat`、`analyze_sentence_breakdown`
`server/tools.py`	`check_syllabus_vocab`、`add_to_vocab_book`、`get_report`
`server/prompts.py`	`SYSTEM_PROMPT`

5.3 长难句解析模块

5.3.1 模块职责

把"每个词都认识、连起来不理解"的句子拆成可复述的结构：篇章与句子级联选择、原句结构着色、五步拆解、结构树、大纲词核验、真题溯源、一键加词。

5.3.2 输入项

输入	来源	说明
篇章列表	`GET /api/passages`	只含已发布篇章
句子列表	`GET /api/passages/{id}/sentences`	含 `word_count`、`is_long`
句子拆解	`GET /api/sentences/{id}/breakdown`	命中缓存直接返回，未命中调用模型
工具链与词表	`POST /api/workflow/sentence`	`vocabulary`、`citations`、`tool_trace`
用户加词动作	点击"加入单词本"	触发 `AppStore.addWord`

5.3.3 输出项

输出	形式
原句	按 `segments` 着色，附"结构标记"图例
五步 Tab	1 找主干、2 剥离从句、3 划分修饰、4 逐层翻译、5 标注语法点
结构树	整句 → 从句/修饰的层级树
大纲词卡片	等级、释义、熟词僻义、频次、来源、加词按钮
真题溯源	当前句 + 同篇切片
工具链	`check_syllabus_vocab` → `search_past_exams`

5.3.4 核心算法与处理逻辑

1. 加载篇章列表，默认选第一项；加载该篇句子，默认选词数最多的句子。
2. 并行请求 `getSentenceBreakdown` 与 `runSentence`。
3. `breakdown` 未命中缓存时，后端用 `analyze_sentence_breakdown` 生成；若模型不可用，返回工具兜底结构：`main` 为整句，`clauses/modifiers` 为空，`tree` 只有一个"整句"节点，`mainNote` 明确说明"当前为工具兜底拆解"。
4. 五步 Tab 的可用性：第一步始终可用；若 `isToolFallback`（无从句、无修饰、无语法点），后四步禁用或显示空态，避免展示伪造结构。
5. 加词时调用 `AppStore.addWord`，携带 `source`（句子出处）与 `context`（原句），实现"词 + 语境 + 出处"三要素归档。
6. 数据变化时（切换篇章/句子）取消过期请求的渲染，避免竞态覆盖。

对应图 6 的时序与图 4a 的长难句分支。

5.3.5 关联源码文件

文件	作用
`web/src/pages/SentencePage.tsx`	页面、级联选择、五步 Tab、结构树、加词
`web/src/components/ToolTrace.tsx`	工具链
`web/src/lib/api.ts`	`getPassages`、`getPassageSentences`、`getSentenceBreakdown`、`runSentence`
`server/app.py`	`/api/passages`、`/api/passages/{id}/sentences`、`/api/sentences/{id}/breakdown`、`/api/workflow/sentence`
`server/workflows.py`	`run_sentence_workflow`、`_top_vocab`
`server/tools.py`	`get_sentence`、`check_syllabus_vocab`、`search_past_exams`
`server/llm.py`	`analyze_sentence_breakdown`

5.4 阅读选项分析模块

5.4.1 模块职责

模拟真实做题而不是"先看答案再解释"：按年份/试卷/Text 筛选篇章，左侧完整原文、右侧题目，作答态与解析态严格分离；提交后逐项给出陷阱类型、原文反证与定位句高亮，并写入练习记录。

5.4.2 输入项

输入	来源	说明
完整题目	`GET /api/questions/full`	含原文、选项、答案、分析、步骤、出处
用户选择	选项按钮	A/B/C/D
提交动作	提交按钮	提交后锁定
用户加词	解析态词条	可选

5.4.3 输出项

输出	形式
三级筛选工具栏	年份、试卷、Text、题数、添加真题（当前无学生导入入口）
左栏原文	按段落展示，独立滚动，提交后滚动到定位句并高亮
右栏题目	题号 Tabs，圆点表示未做/做对/做错；题型、难度、建议时间
作答态	干净四选项 + 提交
解析态	逐项 4px 色条 + 陷阱 Tag + 原文反证 + 正确答案 + 我的选择 + 解题路径 + 工具链 + 出处

5.4.4 核心算法与处理逻辑

1. 加载 `/api/questions/full`，按 `(year, examType, textNumber)` 分组，年份倒序、Text 升序。
2. 两级状态机：`showAnalysis=false` 为作答态，提交后置 `true` 进入解析态。
3. 提交时先构造本地记录并 `addRecord` 镜像，避免刷新前报告不动。
4. 若 `isBasicPractice` 或 `question.reviewStatus === 'practice'`，只调用 `POST /api/records` 保存作答，不生成缺失解析。
5. 否则调用 `runOptions(questionId, selected)`，后端写库并返回 `is_correct`、`selected_trap`、`record`、`tool_trace`。
6. 前端高亮定位句：根据 `sourceSentenceId` 或 `location` 在原文中定位；`location_status !== 'exact_quote'` 时展示"待人工定位"而不强标。
7. 题目切换时使用 `activeQuestionRef` 防止异步响应覆盖当前题。
8. 后端失败时设置 `agentError`，页面提示"只保存在本地"，不阻塞继续训练。

对应图 4a 的选项分支与图 7 的 `record_option_attempt` 消息。

5.4.5 关联源码文件

文件	作用
`web/src/pages/OptionsPage.tsx`	三级筛选、左文右题、两阶段状态机、提交
`web/src/components/Tag.tsx`	陷阱标签配色
`web/src/components/ToolTrace.tsx`	诊断工具链
`web/src/lib/api.ts`	`getQuestionsFull`、`runOptions`、`addRecord`
`server/app.py`	`/api/questions/full`、`/api/workflow/options`、`/api/records`
`server/workflows.py`	`run_options_workflow`
`server/tools.py`	`get_question`、`get_sentence`、`search_past_exams`、`record_option_attempt`
`server/seed.py`	八类陷阱规则数据

5.5 生词本模块

5.5.1 模块职责

把阅读中遇到的词变成可复习资产：展示全部单词、今日到期队列、手动添加；支持掌握度、出处、下次复习与删除；复习时先回忆再显示释义。

5.5.2 输入项

输入	来源	说明
单词列表	`AppStore.words`（`/api/state`）	含 `familiarity`、`nextReview`、`source`、`context`
到期队列	`AppStore.dueWords`	`nextReview <= now`
新增词	表单或外部加词	拼写、释义、出处、语境
复习结果	"记住了/没记住"	布尔

5.5.3 输出项

输出	形式
四个指标	总词数、大纲词数、待复习数、平均掌握度
三个页签	全部单词、今日复习、添加单词
词条	拼写、音标、释义、出处、掌握度、下次复习、删除

输出	形式
复习卡	先显示单词，点击后显示释义，再选择记住/没记住
状态提示	"已连接 FastAPI · 复习记录实时落库"或"本地模式"

5.5.4 核心算法与处理逻辑

1. 添加：前端校验非空与重复（大小写不敏感），通过后乐观插入，再调用 `POST /api/wordbook`；成功后用后端 `id` 与 `nextReview` 替换本地对象。
2. 删除：本地先移除，再 `DELETE /api/wordbook/{spelling}`；失败时置 `connected=false`。
3. 复习：本地先按 `REVIEW_INTERVALS` 更新掌握度与下次复习，再 `POST /api/wordbook/review`；成功后以服务端 `familiarity`、`reviewCount`、`nextReview` 覆盖本地，避免两套算法漂移。
4. 到期判断：`new Date(nextReview).getTime() <= Date.now()`。
5. 复习序列：优先取到期队列；无到期词时取全部单词随机顺序，保证"想练就能练"。
6. 掌握度区间限制在 0–1，前端与服务端同时约束。

5.5.5 关联源码文件

文件	作用
`web/src/pages/WordbookPage.tsx`	三页签、指标、复习卡、删除
`web/src/store/AppStore.tsx`	`addWord`、`removeWord`、`reviewWord`、`dueWords`、`REVIEW_INTERVALS`
`web/src/lib/api.ts`	`getState`、`addWord`、`deleteWord`、`reviewWord`
`server/app.py`	`/api/wordbook`、`/api/wordbook/{spelling}`、`/api/wordbook/review`
`server/tools.py`	`add_to_vocab_book`、`remove_word`、`review_word`、`get_state`
`server/db.py`	`user_vocab_book` 表

5.6 领域知识库模块

5.6.1 模块职责

把"题型、陷阱、熟词僻义、拆解方法"变成可查的领域资产，支撑学生复习方法论与答辩讲解。

5.6.2 输入项

输入	来源	说明
陷阱规则	`GET /api/knowledge` 的 `traps`	八条，含定义、信号、示例、色调

输入	来源	说明
熟词僻义样例	`GET /api/knowledge` 的 `rare_words`	最多 6 条高频词
规模指标	`GET /api/knowledge` 的 `meta`	词条、篇章、题目、陷阱数
题型与方法	`web/src/data/content.ts`	五类题型、两套方法论，静态内容
检索词	搜索框	对题型、陷阱、方法做关键词过滤

5.6.3 输出项

输出	形式
规模指标	词条数、篇章数、题型分类数、陷阱类型数
题型方法论	五类题型：定义、题干信号词、三步解法
陷阱类型	八类：定义、信号特征、配色标识
熟词僻义	单词、等级、基础释义、僻义
拆解方法	长难句五步法、选项分析五步法
检索结果	命中条目数与过滤后的列表

5.6.4 核心算法与处理逻辑

1. 挂载时请求 `/api/knowledge`，失败时使用静态兜底并提示。
2. 陷阱数据来自 `trap_rules` 表，按 `id` 升序；`tone` 字段映射到 `Tag` 的颜色。
3. 熟词僻义只取 `rare_meaning IS NOT NULL` 的词，按 `exam_frequency DESC` 排序，最多 6 条。
4. 题型与方法来自 `content.ts`，不请求后端；检索为纯前端 `includes` 匹配，实时统计命中数。
5. 八类陷阱名称与 `VALID_TRAPS` 保持一致，保证知识库、导入分析、报告统计三处同一套分类。

5.6.5 关联源码文件

文件	作用
`web/src/pages/KnowledgePage.tsx`	页面、检索、题型/陷阱/熟词僻义
`web/src/data/content.ts`	`QUESTION_TYPES`、`TRAP_TYPES`、`METHODOLOGY`、`COMPARISONS`
`web/src/lib/api.ts`	`getKnowledge`
`server/app.py`	`/api/knowledge`
`server/tools.py`	`get_trap_rules`、`get_rare_words`、`get_meta`
`server/seed.py`	`TRAP_RULES` 八条数据

5.7 真题库与后台导入模块

5.7.1 模块职责

两部分职责：

1. **学生端资料库**：浏览大纲词库与已发布真题篇章，检索、筛选、分页，一键加词，跳转训练或拆解。
2. **团队后台导入**：把 JSON 草稿或 PDF/文本结构化后写入 SQLite，经校验、审核、发布后对学生可见。

5.7.2 输入项

输入	来源	说明
词库检索	`GET /api/vocab`	`q`、`level`、`sort`、`offset`、`limit`
篇章列表	`GET /api/passages`	已发布篇章摘要
加词动作	资料库行内按钮	`AppStore.addWord`
题库 JSON 草稿	`scripts/question_bank.py`	含 passage、questions、options、answer
PDF / 原始文本	`scripts/auto_import.py` 或旧接口	当前 HTTP 接口 410，CLI 为首选
答案序列	维护者输入	`21-25 BCDAB`、`BCDA`、`21:B,22:C` 等
审核动作	`transition`	review / publish / publish-practice / withdraw

5.7.3 输出项

输出	形式
词库表格	单词、音标、释义、等级、频次、加入按钮
分页控件	上一页/下一页/页码/跳转，50 条每页
篇章卡片	年份、试卷、Text、预览、词数、句数、题数
跳转	"去做题"带筛选参数到 `/options`；"拆解句子"到 `/sentence`
入库结果	`passage_id`、`question_ids`、`answer_source`、`review_status`、警告列表
状态变更	返回 `publication_status`、`revision`、审核人、发布时间

5.7.4 核心算法与处理逻辑

A. 学生端资料库

1. 词库搜索 300ms 防抖，请求 `browseVocab`；`sort` 支持频次与字母序。
2. 已加入单词本的词在按钮上显示"已在单词本"并禁用。
3. 篇章卡片按年份倒序、Text 升序；点击跳转时携带 `year/examType/textNumber` 查询参数。
4. 分页使用服务端 `offset/limit`，`total` 用于页码计算。

B. 导入校验链（`exam_import.py`）

1. `expected_numbers(text_number)`：Text 1–4 映射 21–25、26–30、31–35、36–40；超出范围抛 `ImportValidationError`。
2. `parse_answer_key(value, numbers)`：支持纯字母序列、范围前缀、显式题号映射；题号范围必须与当前 Text 一致；不接受夹杂垃圾字符。
3. `normalize_passage(text)`：统一换行，按空行分段，段内压缩空白。
4. `validate_extraction(extraction, text_number, answer_key)`：校验五题齐全、题号保留原卷号、四选项非空、每题唯一正确、答案来源合法；不静默截断或顺延。
5. `split_passage(passage)`：按段落与句末标点切句，记录 `paragraph`、`sentence_index`、`char_start`、`char_end`。
6. `locate_quote(quote, passage, sentences)`：逐字匹配引文；唯一命中才绑定 `source_sentence_id` 并置 `location_status=exact_quote`，否则保留引文并置 `unverified`。

C. 写入与原子性（`app.py`）

1. 先查重：`_auto_duplicate` 按 `(year, exam_type, text_number)` 且 ID 以 `auto_` 或 `custom_` 开头；`_manual_duplicate` 按文章全文与题干。
2. 模型调用（抽取、批量分析）在写事务之外完成，避免长时间持锁。
3. 写入时 `BEGIN IMMEDIATE`，持锁后再次查重，防止并发重复。
4. 依次写 `exam_passages` → `exam_sentences` → `exam_questions`，任一失败整体回滚。
5. 题目 `review_status` 初始为 `pending`，`answer_source` 记录 `provided` / `inferred` / `manual`。

D. 审核发布状态机（`question_bank.py`）

```
draft --review--> reviewed --publish--> published
draft --publish-practice--> published（题目 review_status=practice）
published --withdraw--> draft
```

- `review` 前执行 `readiness`：检查元数据、解析完整性、证据绑定、题目数量。
- `publish` 前要求 `publication_status='reviewed'` 且 `reviewed_by` 非空。
- `publish-practice` 只执行 `practice_readiness`，允许基础练习，但不声称解析已审核。
- 每次变更 `revision + 1` 并写 `bank_events` 快照。

E. 快照机制（`scripts/`）

- `export_demo.py` 导出 `demo_snapshot.json`，只含 `auto_` / `bank_` / `custom_` 篇章与句子/题目/拆解缓存，不含个人记录、密钥、审核历史。
- `load_demo.py` 恢复快照，内容为草稿，不直接开放。
- `setup_team.py` 在全新数据库恢复快照、校验 68 篇/340 题、外键与完整性，并开放基础练习；已有数据库时拒绝覆盖。

5.7.5 关联源码文件

文件	作用
`web/src/pages/LibraryPage.tsx`	词库表、篇章卡片、分页、跳转
`web/src/components/ExamImportForm.tsx`	导入表单组件（存在但未挂载）
`web/src/lib/api.ts`	`browseVocab`、`getPassages`、`importExam`、`autoImportExam`
`server/app.py`	`/api/vocab`、`/api/vocab/search`、`/api/passages`、`/api/exams/*`
`server/exam_import.py`	校验、切句、定位
`server/question_bank.py`	批量导入、状态机、解析生成、导出
`server/llm.py`	`extract_exam_text`、`analyze_imported_questions_batch`
`server/scripts/question_bank.py`	CLI 入口
`server/scripts/setup_team.py`	快照恢复与校验
`server/scripts/load_demo.py`	草稿恢复
`server/scripts/export_demo.py`	快照导出

5.8 学情诊断报告模块

5.8.1 模块职责

把练习数据变成可解释的薄弱画像：总练习、正确数、正确率、待复习词；五类题型正确率；薄弱题型；近 7 天练习量；自动建议。

5.8.2 输入项

输入	来源	说明
练习记录	`AppStore.records`	来自 `/api/state`
单词本	`AppStore.words`	用于计算待复习数
题型分类	`QuestionType` 五类	主旨、细节、推理、词义、态度

5.8.3 输出项

输出	形式
四个总览指标	总题数、正确数、正确率、待复习词
题型正确率	五条进度条

输出	形式
薄弱环节	`accuracy < 0.7` 且 `total > 0`，按正确率升序
备考建议	1–4 条：薄弱题型、到期词、样本量提醒
近 7 天练习量	柱状图 + 本周合计
最近练习	最近 8 条

5.8.4 核心算法与处理逻辑

1. 前端按 `questionType` 分组统计 `total` 与 `correct`，计算 `accuracy = correct / total`。
2. 薄弱题型筛选条件：`total > 0 && accuracy < 0.7`，按 `accuracy` 升序。
3. 近 7 天：以本地日期为基准，统计每天 `records` 的条数，生成 7 个柱。
4. 建议生成： - 有薄弱题型：`"{类型}正确率 X%，建议每天精做 2–3 道同类题并复盘排除理由。"` - 有到期词：`"有 N 个单词到期，先完成无提示回忆，再开始新的阅读。"` - 总题数 < 30：`"当前累计 N 题，样本还不够大；建议累计 30 题以上再做稳定薄弱分析。"`
5. 无记录时展示空状态，引导去做第一题。
6. 说明：页面使用前端聚合，不请求 `/api/report`；后端 `/api/report` 使用相同口径但为独立实现（技术债 P-10）。

5.8.5 关联源码文件

文件	作用
`web/src/pages/ReportPage.tsx`	指标、题型进度条、薄弱项、柱状图、建议
`web/src/components/ProgressBar.tsx`	正确率进度条
`web/src/store/AppStore.tsx`	`records`、`words`、`dueWords`
`server/app.py`	`/api/report`（页面未使用）
`server/tools.py`	`get_report`

6 系统出错处理与安全保密设计

6.1 LLM 接口超时、限流降级策略

6.1.1 分层降级模型

系统把 LLM 失败分为五级，逐级降级，保证任何级别失败都不导致页面不可用：

级别	触发条件	系统行为	用户可见结果
L0	未配置 `LLM_API_KEY`	`is_configured()` 返回假，所有生成函数直接返回 `None`	页面正常显示确定性解析与工具链，标识"确定性结果"
L1	网络超时、连接失败	捕获异常并记录 `logger.warning`，返回 `None`	页面正常显示确定性解析与工具链，标识"确定性结果"
L2	401/403/429/5xx	捕获异常并记录 `logger.warning`，返回 `None`，直接回退；**不做自动退避重试**	页面正常显示确定性解析与工具链；429 时用户可稍后重试
L3	返回非 JSON 或 JSON 结构非法	`_extract_json` 或字段校验返回 `None`	页面正常显示确定性解析与工具链，标识"确定性结果"
L4	JSON 合法但字段越界	白名单降级：题型→细节题、陷阱→待分析	页面可展示，但标注"待分析/请复核"

6.1.2 超时参数

调用	超时	说明
`polish_chat`	50s	对话开放讲解
`analyze_imported_question`	50s	单题分析
`extract_exam_text`	90s	整篇抽取
`analyze_imported_questions_batch`	120s	批量分析 5 题
`analyze_sentence_breakdown`	120s	五步拆解

6.1.3 限流现状与建议

现状：项目未实现令牌桶、请求队列、指数退避或并发上限。遇到 429 时直接回退确定性答案，不重试。导入任务为同步请求，可能持续 20–60 秒，多个并发导入会同时占用模型配额。

建议（未实现，不得在文档中宣称已具备）：

1. 对 `429` 与 `5xx` 增加最多 2 次指数退避重试。
2. 引入进程内信号量限制并发 LLM 调用数。
3. 导入改为异步任务队列，返回任务 ID，前端轮询进度。
4. 增加调用耗时与失败率指标，接入可观测性平台。

6.1.4 快照与演示数据在故障中的定位

必须区分两类"回退"：

机制	触发场景	作用	是否自动
确定性结果回退	LLM 未配置、超时、限流、JSON 非法	用工具层已有事实生成回答，保证功能可用	自动

机制	触发场景	作用	是否自动
题库快照恢复	新环境部署、数据库丢失、换机演示	从 `demo_snapshot.json` 重建 68 篇/340 题/1295 句与 6 份拆解缓存	手动执行脚本
草稿快照恢复	需要重新审核题库	`load_demo.py` 恢复为草稿，不直接开放	手动
前端本地兜底	后端不可用	`localStorage["kaoyan-studio-v1"]` 展示已有单词与记录	自动

结论：`demo_snapshot.json` 是**数据级灾备与交付机制**，不是 LLM 故障的自动回退。LLM 故障的自动回退是"确定性工具结果"，两者不可混为一谈。设计文档与答辩表述必须保持这一区分。

6.1.5 前端降级表现

场景	前端行为
`getState` 失败	`connected=false`，右上角显示"本地模式"
长难句拆解失败	展示兜底结构与"工具兜底拆解"说明，不伪造语法树
选项工作流失败	保留本地作答镜像，提示"只保存在本地"
对话失败	提示检查 FastAPI，不使用写死的假答案
资料库请求失败	展示错误空态与重试按钮

6.2 数据校验、异常恢复与备份维护设计

6.2.1 输入校验

层次	校验手段	覆盖范围
HTTP 层	Pydantic `Field` 约束	长度、范围、正则、必填
路径层	FastAPI 路径参数	题目/句子不存在返回 404
语义层	服务端二次校验	`user_answer` 必须是 A/B/C/D；导入四选项齐全；题号必须匹配 Text
导入层	`validate_extraction`	五题齐全、唯一正确、答案来源、原文完整性
数据库层	`NOT NULL`、`UNIQUE`、`CHECK`（间接）、外键	词条唯一、拼写唯一、陷阱名唯一、句子/题目归属
前端层	表单非空、重复词、重复提交禁用	单词本、复习、对话

6.2.2 事务与一致性

1. `db.connect()` 每次请求创建独立连接，`check_same_thread=False`，请求结束关闭。
2. schema 首部启用 `journal_mode=WAL` 与 `foreign_keys=ON`，提升读写并发并保证外键生效。

3. 导入写入使用 `BEGIN IMMEDIATE`，持写锁后二次查重，失败整体回滚。
4. 工具内的写操作（加词、复习、记录）单独 `commit()`，保持短事务。
5. `question_bank.transition` 在事务内校验 `expected_revision`，防止并发覆盖；每次变更 `revision + 1` 并写 `bank_events`。

6.2.3 异常恢复

异常	恢复手段
进程崩溃	SQLite WAL 自动恢复未提交事务
导入中途失败	事务回滚，不留下半篇数据
模型失败	确定性回退，不写坏数据
schema 升级	`migrate_schema` 幂等补列，不重写历史数据
数据库损坏	用 `demo_snapshot.json` + `setup_team.py` 重建；个人记录需另行备份
前端缓存异常	`localStorage` 解析失败返回 `null`，回退到 seed 数据
重复提交	前端提交后禁用按钮；后端唯一约束与应用层查重兜底

6.2.4 备份与维护

维护项	操作	频率	注意
数据库备份	复制 `server/data/app.db`（WAL 模式下同时复制 `-wal`、`-shm` 或先执行 checkpoint）	每次演示前/版本交付前	`*.db` 已被 `.gitignore` 忽略，不提交
题库快照导出	`python scripts/export_demo.py`	题库变更后	不含个人记录、密钥、审核历史
快照恢复	`python scripts/setup_team.py`	新环境	已有数据库时拒绝覆盖
草稿恢复	`python scripts/load_demo.py`	重新审核前	内容为草稿，学生端不可见
词频重算	`seed.py:_count_frequencies`	导入新真题后	口径为当前切片语料内出现次数
拆解缓存清理	删除 `sentence_breakdowns` 对应行	解析质量抽检后	删除后下次访问重新生成
依赖升级	按 `requirements.txt` / `package.json` 更新	迭代前	升级后必须回归导入、工作流、页面

6.2.5 安全保密设计

安全项	措施	残余风险
API Key 保护	只存 `server/.env`；`.gitignore` 忽略 `.env`、`.env.*`；前端无 Key	开发机泄露、终端历史泄露
密钥不入库	数据库与快照不含 Key	备份文件被拷走

安全项	措施	残余风险
数据库保护	`*.db`、`*.sqlite` 被忽略	本地磁盘未加密
PDF 版权	`*.pdf` 被忽略；不随仓库分发；全文只进本地 DB	使用者需自行确认授权
CORS	开发期 `allow_origins=["*"]`	生产必须收敛白名单（P-11）
鉴权	**无登录、无权限控制**	不能公网部署（P-01）
SSRF	HTTP 导入已 410；`_download_pdf_text` 仍存在	若恢复接口需加白名单与内网拦截（P-04）
注入	SQL 使用参数化查询；动态拼接仅用于固定白名单排序与 `WHERE` 片段	排序字段已限定 `freq/alpha`
XSS	React 默认转义；未使用 `dangerouslySetInnerHTML`	模型返回的文本按纯文本渲染
上传大小	PDF 30MB、Text 50000 字符、题干 2000 字符	需在网关层再加限流
日志	不记录 Key；仅 `logger.warning` 记录异常类型	无请求 ID（P-12）
数据隔离	单用户，无 `user_id`	多用户场景数据互相可见（P-01）

6.2.6 安全设计结论

当前系统满足**本地单用户演示**场景的安全需求：密钥不在前端、数据库不随仓库分发、SQL 参数化、模型不能改写事实。但它**不满足公网多用户部署**的安全要求：无鉴权、无隔离、CORS 过宽、导入 URL 缺 SSRF 防护。任何对外表述都不得声称系统已具备生产级安全性。

附录 A 接口清单总表

编号	方法	路径	分组	状态
API-00	GET	`/`	基础	启用
API-01	GET	`/api/health`	基础	启用
API-02	GET	`/api/meta`	基础	启用
API-03	GET	`/api/state`	状态	启用
API-04	GET	`/api/sentences`	内容	启用
API-05	GET	`/api/passages`	内容	启用
API-06	GET	`/api/passages/{passage_id}/sentences`	内容	启用
API-07	GET	`/api/sentences/{sentence_id}/breakdown`	内容	启用
API-08	GET	`/api/questions`	内容	启用
API-09	GET	`/api/knowledge`	内容	启用
API-10	POST	`/api/workflow/sentence`	工作流	启用

编号	方法	路径	分组	状态
API-11	POST	`/api/workflow/options`	工作流	启用
API-12	POST	`/api/workflow/chat`	工作流	启用
API-13	GET	`/api/wordbook`	单词本	启用
API-14	POST	`/api/wordbook`	单词本	启用
API-15	DELETE	`/api/wordbook/{spelling}`	单词本	启用
API-16	POST	`/api/wordbook/review`	单词本	启用
API-17	POST	`/api/records`	记录	启用
API-18	GET	`/api/report`	报告	启用
API-19	GET	`/api/questions/full`	内容	启用
API-20	POST	`/api/exams/import`	导入	停用，410
API-21	POST	`/api/exams/auto-import`	导入	停用，410
API-22	GET	`/api/agent/runs`	日志	停用，410
API-23	GET	`/api/vocab/search`	词库	启用
API-24	GET	`/api/vocab`	词库	启用
API-25	POST	`/api/tools/check-vocab`	工具	启用
API-26	GET	`/api/tools/search-exams`	工具	启用

附录 B HTTP 状态码使用约定

状态码	语义	本项目触发场景	响应体
200	成功	绝大多数查询与工作流成功	业务 JSON
400	请求错误	`check-vocab` 既无 `text` 也无 `words`；PDF URL 非 http(s)；下载失败；未配置 Key 的自动导入	`{"detail": "..."}`
404	资源不存在	句子/题目/篇章不存在或未发布；删除不存在的单词；复习不存在的单词	`{"detail": "..."}`
409	冲突	重复导入同一真题	`{"detail": {"code": "duplicate_exam", ...}}`
410	资源已永久迁移	三个已停用接口	`{"detail": "题库管理已迁移..."}`
422	语义校验失败	Pydantic 字段越界；答案非 A/B/C/D；导入校验失败	`{"detail": [...]}` 或自定义 `issues`
500	服务器内部错误	未捕获异常、数据库错误	FastAPI 默认错误体

附录 C 源码与文档索引

C.1 后端源码

文件	职责	本文档主要引用章节
`server/app.py`	FastAPI 实例、26 个 `/api` 路由、请求模型、连接注入	2.3、3.4、4.3、5.1 ~ 5.8
`server/db.py`	10 张表 schema、连接、迁移、初始化	2.2、2.5、5.7、6.2
`server/seed.py`	词库导入、示例文章、八条陷阱、演示状态、词频统计	3.3、5.4、5.6
`server/tools.py`	四个核心工具、查询、状态、报告、词库浏览	3.3、5.1 ~ 5.8
`server/workflows.py`	三条工作流、意图路由、上下文、运行日志	3.4、5.2、5.3、5.4
`server/llm.py`	DeepSeek 客户端、JSON 清洗、五类生成函数	3.2、4.2、6.1
`server/prompts.py`	`SYSTEM_PROMPT`	3.2
`server/exam_import.py`	题号映射、答案解析、归一化、校验、切句、定位	5.7、6.2
`server/question_bank.py`	批量导入、状态机、解析生成、导出、就绪检查	2.5、5.7、6.2
`server/scripts/setup_team.py`	快照恢复与校验	2.5、6.1、6.2
`server/scripts/load_demo.py`	草稿恢复	6.1、6.2
`server/scripts/export_demo.py`	快照导出	6.1、6.2
`server/scripts/question_bank.py`	建库 CLI	2.5、5.7
`server/scripts/auto_import.py`	历史导入 CLI	5.7
`server/tests/`	迁移、导入、题库测试	2.5

C.2 前端源码

文件	职责	本文档主要引用章节
`web/src/App.tsx`	8 路由 + 404	4.1、5.1 ~ 5.8
`web/src/main.tsx`	应用入口	2.2
`web/src/types.ts`	`Word`、`StudyRecord`	3.3、5.5、图 5
`web/src/lib/api.ts`	唯一 API 客户端与类型	4.3、5.1 ~ 5.8
`web/src/lib/format.ts`	相对时间等格式化	5.1、5.8
`web/src/store/AppStore.tsx`	全局状态、乐观更新、离线兜底	2.3、5.1、5.5、5.8
`web/src/pages/DashboardPage.tsx`	仪表盘	5.1

文件	职责	本文档主要引用章节
`web/src/pages/ChatPage.tsx`	对话	3.1、5.2
`web/src/pages/SentencePage.tsx`	长难句	5.3
`web/src/pages/OptionsPage.tsx`	阅读训练	5.4
`web/src/pages/WordbookPage.tsx`	单词本	5.5
`web/src/pages/KnowledgePage.tsx`	领域知识库	5.6
`web/src/pages/LibraryPage.tsx`	资料库	5.7
`web/src/pages/ReportPage.tsx`	学习报告	5.8
`web/src/components/ToolTrace.tsx`	工具链渲染	3.4、5.2、5.3、5.4
`web/src/components/Layout.tsx`	导航、标题、连接状态	4.1
`web/src/components/ExamImportForm.tsx`	导入表单（未挂载）	5.7
`web/src/components/StatCard.tsx`	指标卡	5.1
`web/src/components/ProgressBar.tsx`	进度条	5.8
`web/src/components/Tag.tsx`	标签与配色	5.4、5.6
`web/src/components/Toast.tsx`	全局提示	4.1、5.5
`web/src/components/EmptyState.tsx`	空状态	4.1、5.1 ~ 5.8
`web/src/components/FeatureCard.tsx`	功能入口卡	5.1
`web/src/components/PageHero.tsx`	页面头部	4.1
`web/src/components/SectionHeader.tsx`	区块标题	4.1
`web/src/data/content.ts`	题型、陷阱、方法论、对比场景	5.6
`web/src/data/seeds.ts`	前端 seed 数据	5.1、5.5
`web/src/index.css`	主题变量、动画、滚动条	4.1

C.3 数据资产

文件	规模	说明
`server/data/kaoyan_vocab.txt`	9,602 非空行，按词头去重 5,047	公共词库，BSD-3-Clause；README 实测口径为 5,064 条
`server/data/rare_meanings.json`	27 条	熟词僻义；文档历史口径称 33 条，以文件为准
`server/data/exam_slices.json`	2 篇 / 8 句 / 3 题	内置示例文章
`server/data/demo_snapshot.json`	68 篇 / 1,295 句 / 340 题 / 6 份拆解	共享题库快照，不含个人记录与密钥

文件	规模	说明
`server/data/LICENSE-vocabulary`	—	词库许可证
`server/data/app.db`	运行时生成	10 张表，已被 `.gitignore` 忽略

C.4 设计文档索引

文档	与本文档的关系
`docs/需求文档.md`	需求输入，本文档第 2.1 节回顾
`docs/ARCHITECTURE.md`	架构与接口概览，本文档第 2、4 章细化
`docs/PROJECT-DOSSIER.md`	设计取舍与技术债，本文档 2.5.2 引用
`docs/概要设计-架构与UML图集.md`	图 1～图 7，本文档各章引用
`docs/数据库设计说明书.md`	10 张表的字段字典与 ER 图，本文档 5.7、6.2 引用
`docs/LOCAL-SETUP.md`	环境与启动，本文档 2.2 引用
`docs/题库建库与发布操作说明.md`	导入、审核、发布流程，本文档 5.7 引用

文档结束。

> 本说明书所有模块名、类名、函数名、表名、字段名均取自代码基线 `main @ 0688e14`。凡历史文档与代码不一致处，已在正文标注差异来源并以代码为准。